AEGIS

Autonomous Engineering, Generation, Intelligence & Self-Repair System

Detailed Phase-by-Phase Implementation Plan

Phase 0 output — greenfield build. Derived from *AUTONOMOUS SOFTWARE ENGINEERING.pdf* (the Specification, Sections 0–56).

Covers strategic positioning and the wedge, a capability spike and walking-skeleton delivery strategy with kill gates, cost / latency economics, and trust / governance — then Phase 0 and Phases 1–28 (Phase 1 is the walking-skeleton checkpoint), each with objective, deliverables, design decisions, implementation steps, phase-wise testing (unit / integration / acceptance / regression), quality gates, 16 objective metrics, risks, and effort sizing. Appendices: traceability matrix, milestones and critical path, the 30-point acceptance contract, the absolute-rules audit, the kill-criteria decision gates, and a cost-model worksheet.

Plan version 1.0

AEGIS — Detailed Phase-by-Phase Implementation Plan

Autonomous Engineering, Generation, Intelligence & Self-Repair System

Derived from: `docs/AUTONOMOUS SOFTWARE ENGINEERING.pdf` (the Specification)

Plan version: 1.0 · Status: Phase 0 output · Target: greenfield build from scratch

1. Document Control

Field	Value
Document	AEGIS Implementation Plan (phase-by-phase, with phase-wise testing)
Source of truth	The Specification PDF in `docs/` (Sections 0–56)
Scope of this document	Planning only. Describes what each phase builds, how it is tested, and its exit gates. No product code is written by this document.
Owning phase	Phase 0 — Greenfield Architecture & Planning (Specification Section 46)
Companion documents (produced during Phase 0 execution)	`AEGIS_ARCHITECTURE.md`, `TECH_STACK.md`, `DATA_MODEL.md`, `SECURITY_MODEL.md`, `MVP_DEFINITION.md`, `AI_AGENT_DESIGN.md`, `METRICS.md`, `EXECUTION_MODEL.md`, `REPOSITORY_ANALYSIS.md`, and this plan
Regeneration	`python scripts/build_plan_pdf.py` renders this Markdown to `docs/AEGIS_IMPLEMENTATION_PLAN.pdf`

1.1 Purpose

Give an engineering team (human or autonomous) a concrete, ordered, testable path from an empty repository to a verified end-to-end AEGIS system that satisfies every item in Specification Section 54 (Final Acceptance Criteria) and violates none of Section 55 (Absolute Rules).

1.2 How to use this plan

1. Execute Phase 0 first: produce the ten architecture documents and the Architecture Decision Records (ADRs) listed in Section 8 of this plan.
1. Then execute Phases 1–28 in order (Phase 1 is the walking-skeleton checkpoint). A later phase may only start when every prior phase it depends on has passed its quality gates.
1. A phase is complete only when its deliverables are **IMPLEMENTED + EXECUTED + TESTED + INTEGRATED + VERIFIED** (Specification Section 47). Writing code is not completion.
1. Every phase below carries an explicit **Phase-wise testing** block (Unit / Integration / Acceptance / Regression, plus E2E or Benchmark where relevant). Those tests are part of the phase deliverable, not an afterthought.
1. Cross-cutting rules (evidence-backed AI conclusions, bounded repair loops, reversible patches, no untrusted host execution, no fabricated results) apply to **every** phase; see Section 4.

1.3 Effort sizing legend

Relative sizing, not calendar estimates: **S** ≈ small, **M** ≈ medium, **L** ≈ large, **XL** ≈ largest / highest risk. The critical-path phases are 8, 9, 11, 13, 20, 21.

---

2. Executive Summary

AEGIS accepts a real software-engineering task (bug, feature, refactor, requirement) plus a target repository and autonomously drives it through a single connected pipeline to a verified change:

```
ISSUE / REQUIREMENT
  -> Task Normalization
  -> Repository Ingestion
  -> Repository Understanding
  -> Codebase Indexing / Code Graph
  -> Issue -> Code Mapping
  -> Dependency + Impact Analysis
  -> Engineering Plan
  -> Plan Validation
  -> Implementation (real file changes)
  -> Test Generation
  -> Secure Execution (Docker sandbox)
  -> Failure Detection
  -> Root-Cause Analysis
  -> Autonomous Debugging + Repair (bounded)
  -> Regression Testing
  -> Change Review
  -> Security + Scope Check
  -> Patch Risk + Confidence
  -> Verification
  -> Commit / PR Generation
  -> Engineering Memory
  -> Final Result
```

Every stage consumes the **typed, schema-validated output** of the previous stage. There are no disconnected AI modules. Execution results always outrank AI assumptions. Every important AI conclusion carries evidence (file, symbol, line, test, commit, execution result). Every score has an explicit, deterministic, versioned formula. Every autonomous action is observable and reversible.

MVP language target: Python repositories. The architecture leaves room for JavaScript, TypeScript, Java, Go, and C++ later, but the plan does not claim multi-language support until it is implemented and tested.

"Done" for the whole system = the 30 criteria in Specification Section 54 each pass with linked test/artifact evidence, demonstrated on the controlled acceptance repository built in Phase 24 and measured by the benchmark framework in Phase 25.

2.1 Strategic thesis

Raw model capability is not the moat — it is a rapidly commoditizing input shared with every competitor. AEGIS wins on **trust per change**: a team can hand AEGIS a task and receive not just a diff but a verifiable, auditable, reproducible engineering record proving the change is correct, in-scope, and safe. The three durable assets are:

1. **Verifiable autonomy** — every change ships with an evidence trace, deterministic risk and confidence scores, an executed test record, and a verification verdict that a human can audit in minutes instead of re-reviewing from scratch.
1. **Repository memory moat** — a persistent, per-repository knowledge graph and engineering-memory layer that compounds: the hundredth task on a codebase is cheaper, faster, and more accurate than the first. A competitor starting cold on that codebase cannot match it.
1. **Deterministic replay** — given the same inputs and recorded model/seed metadata, a task re-runs to the same patch. This is what makes AEGIS acceptable in regulated, security-sensitive, and large-legacy environments where opaque AI commits are a non-starter.

2.2 The wedge

- **Primary user:** engineering teams on large, long-lived Python codebases who currently cannot

  accept autonomous changes because they cannot audit them — regulated industries, security- sensitive products, platform/infra teams, and enterprises with heavy change-control.

- **Primary job:** convert a well-specified issue into a *review-ready, evidence-backed* change,

  cutting human review time and de-risking the "AI wrote this — now what?" gap.

- **Beachhead:** the controlled-repository workflow (Phase 24) generalized to a customer's own

  service repos, sold on review-time reduction and audit completeness, not on autonomy theater.

2.3 Non-goals

- Not a general IDE assistant, chat companion, or autocomplete. AEGIS is a task-in / verified-

  change-out pipeline (Specification Section 52).

- Not chasing raw public-benchmark state-of-the-art. AEGIS optimizes verified-change quality,

  false-complete rate, cost per verified task, and auditability — even when that shows a lower headline resolution rate than marketing-driven competitors.

- Not multi-language at MVP. Python-first, with the abstraction seams for later languages already

  in place but explicitly `UNSUPPORTED` until implemented and tested.

- Not a replacement for human judgment on high-risk changes — those route to `AWAITING_APPROVAL`

  by policy.

2.4 Competitive baseline and kill criteria

The benchmark framework (Phase 25) does not run in a vacuum. It executes the **same task set** through at least two open reference agents (e.g. OpenHands / SWE-agent, and Aider) and reports a relative delta (metric #15). AEGIS must beat those baselines on **verified-change quality and false-complete rate**, not necessarily on raw resolution count.

Explicit kill / pivot criteria are evaluated at the decision gates in Section 7.5. If, after the capability spike, issue->code localization or end-to-end repair success is below the stated floors and cannot be recovered by retrieval/planning redesign within a bounded effort, the plan pauses breadth work and re-scopes rather than building surfaces on a weak core.

---

3. Architecture Overview

3.1 System context

- **AEGIS** is the system being built. It is trusted infrastructure.

- **Target repositories** are external and **untrusted**. Their files, Git metadata, tests, and any

  code AEGIS generates for them are treated as hostile input until proven safe.

- **AI providers** are external services behind an abstraction; their outputs are untrusted until

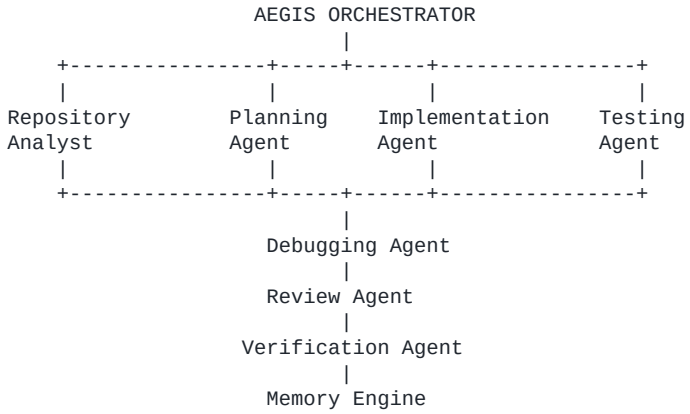
  schema-validated and evidence-checked.

3.2 Component map

Layer	Components
Entry	FastAPI HTTP API, Excel import/export, (optional) GitHub issue intake
Orchestration	Orchestrator, Job queue + worker, Workflow state machine
Agents	Repository Analyst, Planning, Implementation, Testing, Debugging, Code Review, Verification
Engines	Repository intelligence (AST), Code graph (NetworkX), Issue->Code mapping, Impact analysis, Regression selection, Scoring (risk/confidence/health), Reporting
Platform services	AI provider abstraction, Docker sandbox, Git/GitHub clients, Artifact storage, Engineering memory, Observability
Persistence	SQLAlchemy models on SQLite (dev) with a PostgreSQL-compatible schema
Frontend	React + TypeScript + Vite dashboard (14 screens + diff/patch viewer)

3.3 Multi-agent model (Specification Section 4)

A **small** set of specialized agents coordinated by a central **Orchestrator** that owns workflow state. Agents never exchange free text; they exchange typed schemas (Section 4.6 of this plan and Specification Section 20).



3.4 Connectedness invariant

Automated test in Phase 21: for a completed task, assert that each stage's persisted input record is byte-identical to the prior stage's persisted output record (no stage re-derives facts, no stage runs on unstructured text). This invariant is what makes AEGIS a pipeline rather than a bag of prompts.

---

4. Cross-Cutting Foundations

These are decided in Phase 0 and honoured by every later phase.

4.1 Technology stack (Specification Section 44)

Concern	Choice	Notes
Backend language	Python 3.11+	
Web framework	FastAPI + Pydantic v2	OpenAPI, structured errors, validation
ORM / migrations	SQLAlchemy 2.0 + Alembic	SQLite dev, PostgreSQL-compatible schema
Code analysis	Python `ast` (primary), `tree-sitter` (resilience / future languages)	never import or exec target code

Concern	Choice	Notes
Graph	NetworkX in-memory + adjacency tables in DB	betweenness/degree centrality feed risk
Git	GitPython (local history), REST client for GitHub	no shell git with untrusted input
AI	`AIProvider` abstraction: Claude / OpenAI / Local / Mock	schema-validated outputs, retries, budgets
Sandbox	Docker (rootless where possible)	`--network none` default, full resource caps
Testing	pytest, pytest-cov, hypothesis; Vitest + RTL + MSW; Playwright	mock AI by default
Excel	openpyxl (write-only mode for big sheets)	shares domain services with API
Optional ML	scikit-learn	only for justified scoring/ranking calibration
Containerization	Docker + Docker Compose	api, worker, frontend, optional postgres
CI	Lint (ruff/black) + types (mypy) + pytest + coverage gate + frontend build/lint/tsc + Playwright	

Additional technologies require an ADR justifying them. Do not add complexity without cause.

4.2 Proposed project structure (Specification Section 45, refined)

```
aegis/
  backend/
    app/
      api/            # FastAPI routers, one module per endpoint group
      core/           # config, logging, errors, security primitives, limits
      db/             # session, base, migrations wiring
      models/         # SQLAlchemy models (Section 4.4)
      schemas/        # Pydantic schemas incl. the 18 AI output schemas
      orchestration/  # orchestrator, job queue, worker, state machine
      agents/         # repository_analyst, planning, implementation, testing,
                      #   debugging, review, verification
      repository/     # ingestion, workspace, git client, url validation, limits
      analysis/       # python_ast, symbols, imports, project_meta, graph/, impact,
                      #   mapping/ (lexical, semantic, fuse, evidence)
      implementation/ # editor, patcher, scope tracker, rw workspace
      testing/        # generator, selector, regression, catalog
      debugging/      # investigate, rca, hypotheses, repair_loop, guard
      review/         # agent, static_checks, rules, aggregate
      verification/   # agent, criteria evaluators, trace
      scoring/        # confidence, risk, repo_health, model_registry
      memory/         # store, index, retrieve
      github/         # client, provider, pr_builder
      sandbox/        # runner, docker_backend, policy, resource_limits, result_parser
      reporting/      # excel_import, excel_export, report_builder, templates
    tests/            # unit / integration / acceptance / security / perf / e2e
  frontend/
    src/
      pages/ features/ components/ services/ hooks/ types/
  test-repositories/  # controlled acceptance repo + fixtures
  benchmarks/         # datasets, runner, tasks.yaml, report
  docs/               # the ten Phase 0 documents + this plan + guides
  scripts/            # build tools, dev helpers
  docker/             # api.Dockerfile, sandbox.Dockerfile, compose files
  README.md
```

Phase 0 may refine this; any change is recorded in an ADR.

4.3 Workflow state machine (Specification Section 19, 36)

States: `PENDING`, `QUEUED`, `INGESTING`, `ANALYZING`, `PLANNING`, `PLAN_VALIDATION`, `IMPLEMENTING`, `GENERATING_TESTS`, `EXECUTING_TESTS`, `INVESTIGATING`, `REPAIRING`, `REGRESSION_TESTING`, `REVIEWING`, `VERIFYING`, `AWAITING_APPROVAL`, `COMPLETED`, `FAILED`, `CANCELLED`, `PARTIALLY_SUPPORTED`.

From	To (allowed)	Trigger
PENDING	QUEUED, CANCELLED	task run requested / cancelled
QUEUED	INGESTING, CANCELLED	worker picks up job
INGESTING	ANALYZING, FAILED, PARTIALLY_SUPPORTED, CANCELLED	repo cloned / limit hit / error
ANALYZING	PLANNING, FAILED, PARTIALLY_SUPPORTED, CANCELLED	analysis persisted
PLANNING	PLAN_VALIDATION, FAILED, CANCELLED	plan produced
PLAN_VALIDATION	IMPLEMENTING, PLANNING, FAILED, AWAITING_APPROVAL, CANCELLED	approved / revise / high-risk
IMPLEMENTING	GENERATING_TESTS, FAILED, CANCELLED	patch generated
GENERATING_TESTS	EXECUTING_TESTS, FAILED, CANCELLED	tests written
EXECUTING_TESTS	REGRESSION_TESTING, INVESTIGATING, FAILED, CANCELLED	pass / fail
INVESTIGATING	REPAIRING, FAILED, CANCELLED	failure analysed
REPAIRING	EXECUTING_TESTS, REGRESSION_TESTING, FAILED, CANCELLED	candidate applied / budget exhausted (safe stop)
REGRESSION_TESTING	REVIEWING, INVESTIGATING, FAILED, CANCELLED	regression clean / new failure
REVIEWING	VERIFYING, FAILED, CANCELLED	review complete
VERIFYING	COMPLETED, AWAITING_APPROVAL, FAILED, CANCELLED	verified / needs approval / not verified
AWAITING_APPROVAL	VERIFYING, COMPLETED, CANCELLED, FAILED	human decision
COMPLETED / FAILED / CANCELLED / PARTIALLY_SUPPORTED	(terminal)	

All transitions are explicit, guarded, persisted, and emitted to the task timeline.

4.4 Data model (Specification Section 35)

Entities: `Repository`, `RepositorySnapshot`, `RepositoryFile`, `RepositorySymbol`, `Dependency`, `Commit`, `Issue`, `Task`, `TaskStep`, `RepositoryAnalysis`, `CodeMapping`, `ImpactAnalysis`, `EngineeringPlan`, `Implementation`, `Patch`, `TestCase`, `TestExecution`, `Failure`, `Investigation`, `RepairAttempt`, `ReviewFinding`, `RiskAssessment`, `Verification`, `PullRequest`, `EngineeringMemory`, `Job`, `Artifact`, `AuditLog`.

Every table gets: surrogate primary key, foreign keys with explicit `ondelete`, `created_at` / `updated_at` (UTC), a `status` field where a lifecycle exists, uniqueness constraints where natural keys exist (e.g. `RepositorySnapshot(repository_id, commit_sha)`), and indexes on every foreign key plus common query columns (`Task.status`, `Job.state`, `RepositoryFile.snapshot_id, path`). Large or generated blobs (patches, logs, reports, serialized graphs) are stored via the Artifact storage abstraction (Section 4.9), not in wide text columns.

4.5 AI provider abstraction (Specification Section 3.8)

`AIProvider` interface: `complete(prompt, schema, *, timeout, max_tokens, temperature) -> Validated` and `embed(texts) -> vectors` (optional). Implementations: `ClaudeProvider`, `OpenAIProvider`, `LocalProvider`, `MockProvider` (deterministic, used by default in CI). Every implementation provides: structured prompt assembly from templates (never raw string concatenation of untrusted text), JSON-schema-constrained output, one automatic repair round on invalid output then clean failure, retries with exponential backoff, per-call timeout,

token/context budgeting with deterministic truncation and provenance, and request/response logging with secret redaction. The active provider is chosen by environment config; absence of any real provider still yields a working (lower-confidence, rule-based fallback) pipeline.

4.6 AI output schemas (Specification Section 20)

Eighteen Pydantic schemas, each defining required fields, types, enums, a `confidence` field, an `evidence` list, and explicit failure handling: `RepositoryAnalysis`, `IssueAnalysis`, `IssueCodeMapping`, `ImpactAnalysis`, `EngineeringPlan`, `PlanValidation`, `ImplementationResult`, `TestGeneration`, `TestExecution`, `FailureAnalysis`, `RootCauseAnalysis`, `RepairProposal`, `ReviewFinding`, `PatchRiskAssessment`, `PatchConfidence`, `VerificationResult`, `PullRequestDraft`, `EngineeringMemory`. Every schema is validated before its consumer runs; an `Evidence` item is `{kind: file|symbol|line_range|test|commit|dependency|execution, ref: str, detail: str}`.

4.7 AI hallucination control (Specification Section 21)

Every AI conclusion is tagged `FACT`, `INFERENCE`, `HYPOTHESIS`, or `RECOMMENDATION`. Missing information is reported as `UNKNOWN`; weak evidence is `LOW CONFIDENCE`. A `FACT` must reference at least one concrete `Evidence` item that AEGIS can independently re-check. Execution results (sandbox test outcomes) always override AI assertions. AI confidence never substitutes for verification.

4.8 Sandbox threat model and control matrix (Specification Sections 18, 34)

Threat	Control
Malicious repo scripts / build hooks	Docker container, non-root UID, `--cap-drop ALL`, `--security-opt no-new-privileges`, git cloned with `core.hooksPath=/dev/null`
Shell execution / command injection	no `shell=True` anywhere; subprocess allowlist; only `docker` is invoked on the host
Filesystem escape / path traversal	`--read-only` rootfs + `tmpfs` for scratch; only the task workspace bind-mounted `rw`; host-side path-jail util resolves and asserts containment
Network exfiltration / SSRF	`--network none` by default; dependency install is opt-in, host-allowlisted, hash-pinned; GitHub client blocks private IPs and non-allowlisted hosts
Credential / env-var theft	env scrubbed to an allowlist before entering the sandbox; secrets never mounted; logs redacted
CPU / memory exhaustion	`--cpus`, `--memory`, `--memory-swap`, wall-clock timeout with kill
Process spawning / fork bombs	`--pids-limit`, `--ulimit nproc`
Malicious tests / generated code	executed only inside the sandbox; static safety rules (eval/exec/subprocess/secret literals) run first
Container escape	pinned base image by digest, minimal image, seccomp default (custom profile optional), no Docker socket, rootless daemon where available
Supply-chain	pinned + hashed dependencies, `pip-audit`, SBOM, offline by default
Cleanup	container and volume always removed; workspace GC; execution fully logged

If Docker is unavailable, execution phases return `PARTIALLY_SUPPORTED` with a clear reason rather than falling back to host execution. Stronger isolation (gVisor, Firecracker, nsjail) is a documented future option.

4.9 Artifact storage and repository limits (Specification Sections 36, 37)

- **Database:** structured records, relationships, small fields, status.

- **Filesystem / artifact store:** patches, diffs, logs, serialized graphs, reports, workspaces.
- **Temporary workspace:** per-task, isolated, reset-able, garbage-collected.
- Configurable limits: repository size, file count, individual file size, Git history depth,

analysis duration, generated-test count, AI context size, patch-candidate count, sandbox runtime / memory / CPU. Exceeding a limit yields `PARTIALLY_SUPPORTED` with a reason, never a crash and never silent truncation without provenance.

4.10 Scoring algorithms (Specification Sections 12, 13, 14, 16)

All three scores are deterministic, explainable, versioned (`scoring-model v1.0.0`, calibrated in Phase 25), and stored with a per-signal contribution breakdown plus evidence references. Each signal is normalized to `[0, 1]`. Weights within a score sum to `1.0`. Every constant is documented with a rationale; a test asserts the code constants equal the documented table.

4.10.1 Patch Confidence Score (PCS), 0–100 (higher = safer to accept)

Signals (each -> `[0,1]`, higher = better):

Signal	Definition	Weight
targeted_pass	passed / executed targeted tests	0.22
regression_pass	passed / executed regression tests	0.22
review_clean	`1 - min(1, (2*critical + high + 0.4*medium) / 5)`	0.15
coverage	fraction of changed lines covered by executed tests	0.10
scope_clean	`1.0` no violation, `0.5` overridden-with-reason, `0.0` unresolved	0.10
size_fit	`clamp(1 - lines_changed / 400, 0, 1)`	0.06
dep_fit	`clamp(1 - impacted_callers / 25, 0.2, 1)`	0.05
history_stable	`1 - churn_percentile_of_touched_files`	0.04
repair_fit	`clamp(1 - repair_iterations / max_iterations, 0, 1)`	0.04
ai_selfconf	provider-reported confidence, contribution capped	0.02

```
PCS_raw = 100 * sum(weight_i * signal_i)
```

`security_gate = 1.0` if no security review finding >= HIGH, `0.6` if a MEDIUM security finding is open, `0.0` if a HIGH/CRITICAL security finding is unresolved.

```
PCS = round(PCS_raw * security_gate)
```

Hard override: any unresolved CRITICAL review finding, any unresolved scope violation, or any failing regression test caps `PCS` at `40` and sets classification `BLOCKED`.

Classification: `>= 85` HIGH · `70–84` MEDIUM · `50–69` LOW · `< 50` VERY LOW / BLOCKED.

4.10.2 Change Risk Score (CRS), 0–100 (higher = riskier)

Signal	Normalization	Weight
files_changed	`clamp(n / 10, 0, 1)`	0.10
lines_changed	`clamp(loc / 300, 0, 1)`	0.10

Signal	Normalization	Weight
dependency_impact	`clamp(impacted_symbols / 30, 0, 1)`	0.12
public_api_touched	`1.0` if an exported symbol / route changes else `0.0`	0.15
inverse_coverage	`1 - coverage_of_changed_lines`	0.13
historical_churn	churn percentile of touched files	0.08
prior_failures	`clamp(failures_in_area / 3, 0, 1)`	0.10
architectural_centrality	normalized betweenness of touched graph nodes	0.12
complexity_delta	`clamp(added_cyclomatic / 20, 0, 1)`	0.05
security_sensitivity	`1.0` if auth/crypto/exec/serialization/path/network touched else `0.0`	0.05

```
CRS = round(100 * sum(weight_i * signal_i))
```

Thresholds: `0–24` LOW · `25–49` MEDIUM · `50–74` HIGH · `75–100` CRITICAL.

4.10.3 Repository Health Profile (RHP), 0–100 (higher = healthier) and **Task-Specific Risk Profile**

RHP weighted sub-scores: maintainability index (0.25), test coverage (0.25), inverse dependency coupling (0.15), churn stability (0.15), documentation ratio (0.10), CI presence (0.10). Also emits `risky_modules` = top decile by `centrality * churn * inverse_coverage * complexity`. The Task-Specific Risk Profile restricts RHP inputs to the files in the current impact set and is attached to the plan and the verification result.

Every scoring metric documents: metric, purpose, signals, normalization, weights, formula, thresholds, categories, evidence, version (Specification Section 13).

4.11 Human-in-the-loop policy (Specification Section 29)

Per-action policy value: `AUTO`, `REVIEW_REQUIRED`, `BLOCKED`, chosen by configurable rules. Default `AUTO`: analyze, plan, generate tests, run sandbox tests, generate patch (local). Default `REVIEW_REQUIRED`: modifying protected branches, external PR creation, dependency upgrades, database migrations, patches with `CRS >= HIGH` or `PCS < 70`. Default `BLOCKED`: destructive operations, unresolved scope violation, unresolved CRITICAL finding. A `REVIEW_REQUIRED` action parks the task in `AWAITING_APPROVAL`.

4.12 Observability and explainability (Specification Sections 24, 25)

Per task, record: workflow state history, per-agent input/output/duration/errors, AI request metadata (no secrets, no full prompts with untrusted content beyond a redacted digest), tool calls, test executions, patch iterations, verification results. Every task produces an engineering trace answering: **why this file?** (evidence), **why this change?** (requirement mapping), **why this test?** (behaviour covered), **why is this patch safe?** (tests + regression + scope + review). Logs and traces are available through the API and dashboard.

4.13 Cost and latency budget

Autonomy is only useful if it is cheaper and faster than the human review it replaces. Cost and latency are first-class design constraints, not an afterthought.

- **Per-task envelope.** Every task carries a cost budget (priced model spend) and a wall-clock

  budget, both configurable, both enforced by the Orchestrator. Exceeding a budget parks the task in `AWAITING_APPROVAL` or `PARTIALLY_SUPPORTED` with the partial artifacts, never a silent overrun. Target envelope and the assumptions behind it are documented in `docs/COST_MODEL.md` and Appendix F.

- **Model routing.** A `cheap` tier handles task normalization, `task_type` classification,

test-collection triage, lexical re-ranking, and summarization. A `frontier` tier handles planning, implementation edit-operation generation, root-cause analysis, and code review. The router is config-driven and per-stage; routing decisions are logged with the token and cost accounting.

- **Caching.** Prompt/context caching for the repeated repository-context prefix across stages of

  the same task; embedding and lexical indexes are built once per snapshot and reused.

- **Per-stage AI-call budget.** Each stage declares a maximum number of model calls; the repair

  loop additionally has the Phase 14 iteration and wall-clock bounds. The Orchestrator refuses a stage that would exceed its call budget and records the refusal.

- **Deterministic context windowing.** When context exceeds the model limit, truncation is

  deterministic (stable ordering, documented priority: changed symbols > direct neighbours > related tests > history) and every dropped item is recorded as provenance.

- **Early exit.** The repair loop stops when the marginal reduction in failing tests per iteration

  falls below a configured threshold, rather than spending the full iteration budget on diminishing returns.

  Enforced and measured by metrics #13 (cost per verified task) and #14 (latency P50/P95), with CI budget-regression gates in Section 5.7.

4.14 Trust, governance, and deterministic replay

The wedge (Section 2.2) is auditable autonomy. These mechanisms make an AEGIS change acceptable where an opaque AI commit is not.

- **Deterministic replay.** Every task records its full input, the resolved provider/model ids,

  request parameters, and any seeds. A replay re-runs the recorded task and asserts the produced patch (and key intermediate artifacts) match. Non-determinism that cannot be eliminated (e.g. a provider without seed support) is disclosed in the Trust Report rather than hidden. Measured by metric #16 (replay fidelity).

- **Tamper-evident audit log.** `AuditLog` entries are hash-chained (each row stores the hash of

  the previous row plus its own content); every mutating API call and every autonomous action (patch applied, test executed, PR created, approval granted) writes one. A verification endpoint checks chain integrity.

- **RBAC.** API roles: `viewer` (read), `operator` (create/run/cancel tasks), `approver` (resolve

  `AWAITING_APPROVAL`, override scope/risk gates), `admin` (config, providers, policy). Every mutating route declares its required role.

- **Data handling.** Repository contents and issue text are never submitted for provider model

  training; the provider allowlist is explicit configuration; a local/on-prem `LocalProvider` path exists for environments that cannot send code to a third party. The request logger stores only redacted digests of any prompt containing untrusted content.

- **Trust Report.** On every terminal task, AEGIS emits a single `TrustReport` artifact bundling

  the engineering trace, the issue->code evidence, the deterministic risk and confidence scores with their per-signal breakdown, the review findings, the verification verdict, the replay- fidelity result, and the audit-chain reference — the one document a human approver needs to sign off a change quickly.

---

5. Global Testing Strategy

Phase-wise testing is mandatory. This section defines the shared machinery; each phase then lists its specific tests.

5.1 Test layers

Layer	Purpose	Tools
Unit	pure logic, parsers, normalizers, formulas, guards	pytest, hypothesis
Schema / contract	every AI output schema and every API response validates; OpenAPI snapshot	pytest, Pydantic, schemathesis-style checks
Integration	a phase's modules working together against real fixtures (DB, workspace, graph)	pytest, tmp SQLite, fixture repos
Sandbox	Docker isolation, resource caps, escape/exhaustion attempts	pytest + docker-py, `@pytest.mark.docker`
Acceptance	phase deliverable satisfies its Specification Section 49 quality gate on the controlled repo	pytest
Regression	prior phases still pass; golden snapshots stable; seeded-fault detection rates hold	pytest, golden files
E2E	full pipeline through the API and the dashboard	pytest, Playwright
Benchmark	the 16 objective metrics over a curated dataset	`benchmarks/runner.py`

5.2 Determinism and provider policy

- `MockProvider` is the default in CI: canned, schema-valid responses keyed by prompt hash. All unit/integration/acceptance/E2E tests use it.
- Real-provider tests are marked `@pytest.mark.live_ai` and run only when `RUN_LIVE_AI=1` (nightly), never gating a merge.
- Sandbox tests are marked `@pytest.mark.docker`; CI runs them on a Docker-enabled runner. Where Docker is absent, the suite asserts the `PARTIALLY_SUPPORTED` path instead.

5.3 Coverage and quality gates (CI)

- Backend line coverage `>= 85%` overall, `>= 90%` on `core/`, `scoring/`, `sandbox/policy`, `security/`; branch coverage `>= 75%` on those critical modules.
- Agent modules `>= 70%` (mocked provider).
- Frontend: `tsc` clean, ESLint clean, Vitest suites green, Playwright E2E green.
- Lint (ruff), format (black), types (mypy) clean. `bandit`, `pip-audit` clean or triaged with a documented exception.
- No secret pattern in any log or artifact (scanner test).

5.4 Definition of Done (per phase) — Specification Section 47

For each phase: (1) inspect current state, (2) design the smallest complete implementation, (3) implement, (4) run tests, (5) inspect real failures, (6) fix root causes, (7) run regression, (8) verify integration with prior phases, (9) record completion with evidence links, (10) continue. No phase is "done" on scaffolding, TODOs, or unexecuted tests.

5.5 Shared fixtures

- `test-repositories/aegis-acceptance/` — the controlled repo (finalized in Phase 24; a minimal version exists from Phase 3 so earlier phases have something real to run against).
- `test-repositories/fixtures/` — small single-purpose repos: syntax-error file, monorepo layout, poetry vs pep621 vs requirements, aliased imports, dynamic dispatch, network-touching test, fork-bombing test, memory hog, out-of-workspace writer, seeded-secret reader.
- `benchmarks/datasets/` — curated mini-repos with gold mappings and seeded faults, disjoint from the acceptance repo.

5.6 Capability evaluation harness

Capability is measured against external references, not only internal fixtures.

- **Public benchmark subset.** Adopt a fixed subset of real issues in SWE-bench-Lite task format (problem statement + repository at a base commit + gold patch + gold test set). Start with ~30 tasks for the Phase 0 capability spike; grow to a few hundred for Phase 25.
- **Held-out "wild" set.** A separate pool of real open-source issues, never used to tune retrieval, prompts, or weights — used only for final measurement, to detect overfitting to the curated set.
- **Seeded sets.** The seeded-fault and seeded-regression repositories (Section 5.5) drive metrics #5, #6, and #9; the labeled verification set drives metric #10 and the false-complete rate.
- **Reference agents.** The harness can run the same task set through at least two open agents (OpenHands / SWE-agent, and Aider) to produce the competitive delta (metric #15).

5.7 Cost and latency regression gates

- `MockProvider` accumulates per-call token counts; a priced model table converts them to a simulated USD cost per task. CI asserts the median and P95 cost and wall-clock latency for the walking-skeleton E2E and the acceptance E2E stay within documented budget bands.
- A change that pushes a task outside its band fails the build until the band is re-justified and updated with an ADR.

5.8 Determinism and replay tests

- A first-class test layer: run a task with `MockProvider`, persist its full input + resolved model metadata, replay it, and assert the produced patch and the key intermediate artifacts (mapping, plan, diff) are byte-identical.
- Runs on every change to a core-phase module. Feeds metric #16 and the Trust Report replay-fidelity field.

---

6. Objective Metrics (Specification Section 40)

Metrics 1–12 are the Specification's set; 13–16 extend it with the economic, competitive, and audit dimensions the wedge (Section 2) depends on. Each metric: definition, formula, inputs, interpretation, limitations, dataset. Calibrated and reported with real measured values in Phase 25 (`docs/METRICS.md`, `docs/BENCHMARK_RESULTS.md`). Published numbers are always generated by the runner, never hard-coded.

#	Metric	Formula	Inputs	Interpretation / limitation	Dataset
1	Issue->Code Mapping Accuracy	`F1(predicted_files, gold_files)`; also recall@k	mapping output, gold file lists	higher = better localization; sensitive to gold-set granularity	benchmark set + acceptance repo tasks
2	Plan->Implementation Alignment	`(steps_implemented / steps_total) * (1 - unplanned_files / max(1, changed_files))`	plan, final diff, traceability	1.0 = every step done, nothing extra; punishes scope creep	benchmark tasks
3	Test Generation Validity	`tests_valid / tests_generated` (valid = collects + deterministic + has assertions)	test catalog, collect results	low value = unusable generated tests	benchmark tasks
4	Test Pass Rate	`tests_passed / tests_executed`	sandbox execution results	context metric, not a goal by itself	all runs
5	Autonomous Repair Success Rate	`repaired_to_green / tasks_entering_repair`	repair ledger	headline autonomy metric; depends on fault difficulty mix	seeded-fault set
6	Regression Detection Rate	`injected_regressions_caught / injected_regressions_total`	seeded regressions, regression results	measures safety net strength	seeded-regression set
7	Patch Acceptance Rate	`patches_verified_and_scope_clean / patches_generated`	verification + scope results	end-to-end quality; sensitive to task mix	benchmark tasks
8	Scope Compliance	`1 - tasks_with_unjustified_scope_violation / tasks_total`	scope guard events	should be near 1.0	all runs
9	Review Defect Detection	`seeded_defects_flagged / seeded_defects_total`	seeded-defect patches, review findings	measures reviewer power; not a false-positive measure	seeded-defect set
10	Verification Accuracy	`(TP + TN) / total`; also false-complete = `FP / (FP + TN)`	verifier verdicts vs ground truth	false-complete must be ~0	labeled verification set
11	Mean Repair Iterations	`sum(iterations) / tasks_entering_repair`	repair ledger	efficiency; bounded by `max_iterations`	seeded-fault set
12	Task Completion Rate	`tasks_completed_verified / tasks_submitted`	task statuses	overall throughput of verified outcomes	benchmark + acceptance
13	Cost per Verified Task (USD)	`total_priced_model_spend / tasks_verified`	per-call token accounting, priced model table	economic viability vs. human review cost; sensitive to model prices and task mix	benchmark set
14	Task Latency P50 / P95	percentiles of wall-clock from `run` to terminal state	job timestamps	responsiveness; dominated by sandbox and frontier-model calls	all runs

#	Metric	Formula	Inputs	Interpretation / limitation	Dataset
15	Competitive Resolution-Rate Delta	`aegis_verified_scope_clean_rate - best_reference_agent_rate` on the identical task set	AEGIS + reference-agent outcomes	relative standing on verified-change quality, not marketing headline; reference agents evolve	shared benchmark set
16	Deterministic Replay Fidelity	`tasks_replaying_to_identical_patch / tasks_sampled`	recorded task input + model metadata, replay results	audit / regulatory readiness; provider non-determinism lowers the ceiling	benchmark sample

Headline gate. The **False-Complete Rate** (`FP / (FP + TN)` from metric #10 — a task marked `VERIFIED` that is not actually correct) is a named release-gating metric with a hard ceiling of **2%**. Shipping with a higher false-complete rate is not permitted regardless of other scores.

---

7. Phase Roadmap and Delivery Strategy

The Specification's 27-phase order is sound, but executed literally it builds breadth before the core is proven. This section overlays a **capability-first, thin-spine** delivery strategy on top of those phases: prove the hard part, get one real task end-to-end, then thicken and harden, and keep breadth explicitly below an MVP cut line.

7.1 Stage A — Capability spike (pre-build gate)

Before building any product surface, a time-boxed spike (throwaway wiring, no persistence, no API) measures the two capabilities that determine whether the whole idea works:

- **Issue->code localization** — recall@10 on a fixed ~30-task benchmark subset (Section 5.6),

  target floor **>= 0.75**.
- **End-to-end verified fix** — the spike drives a naive map -> plan -> patch -> run-tests loop and

  counts tasks whose gold tests pass afterwards, target floor **>= 0.30** on that subset.

  Outcomes:

- At or above both floors -> proceed to Stage B.
- Below a floor -> redesign retrieval (hybrid weighting, chunking, graph signals) and/or the

  planning prompt, re-run the spike; bounded to two redesign rounds.
- Still below after the bounded effort -> trigger the kill / pivot criteria in Section 7.5

  (re-scope to assisted rather than autonomous, or narrow the task class) before spending on surfaces.

  Recorded in `docs/CAPABILITY_SPIKE.md` as a Phase 0 exit artifact.

7.2 Stage B — Walking skeleton

The narrowest possible end-to-end vertical slice, built as **Phase 1** (Section 9):

```
one real task -> ingest -> map -> plan -> patch -> sandbox test -> repair -> verify -> printed diff
```

No dashboard, no engineering memory, no GitHub, no Excel, no scoring calibration — reduced "just enough" implementations of the spine phases, wired through a minimal Orchestrator. The moment this passes on one real task, the riskiest integration question is answered and there is a regression anchor for everything that follows (until the full controlled repo in Phase 24 replaces it).

7.3 Spine / Thicken / Harden / Deferred

Phase	Role	Note
0 Architecture & planning	Spine	+ capability spike
1 Walking skeleton	Spine	earliest end-to-end proof
2 Project foundation	Spine	skeleton uses a reduced subset
3 Repository ingestion	Spine	
4 Repository intelligence	Spine	
5 Code graph	Thicken	skeleton uses import + name-based edges only
6 Task ingestion	Spine	
7 Issue->code mapping	Spine	the capability that must be strong
8 Impact analysis	Thicken	skeleton uses direct callers only
9 Engineering planning + validation	Spine	
10 Real patch generation	Spine	
11 Test generation	Spine	
12 Secure execution (sandbox)	Spine	highest technical risk
13 Failure investigation	Spine	
14 Autonomous debugging & repair	Spine	
15 Regression intelligence	Thicken	skeleton runs targeted + full suite, no smart selection
16 Code review engine	Thicken	skeleton runs static checks only
17 Risk & confidence engine	Thicken	skeleton emits raw signals, no calibrated score
18 Verification	Spine	
19 Git / GitHub integration	Harden (local) / Deferred (write-automation)	local Git in MVP; PR *creation* deferred
20 Engineering memory	Thicken	the compounding moat; not needed for first proof
21 FastAPI completion + orchestration	Spine	
22 Frontend dashboard	Thicken (task pipeline view) / Deferred (14-screen polish)	
23 Excel / reporting	Deferred	post-MVP
24 End-to-end controlled repository	Spine	
25 Benchmarking + calibration	Thicken (spike harness) / Deferred (productization)	
26 Security hardening	Harden	gating before any external use
27 Performance & reliability	Harden	
28 Documentation & release	Harden	

MVP cut line. The MVP is: Phases 0–19 (0 planning, 1 walking skeleton, 2–19 the sequential build-out through Verification and local Git), 20 (basic), 21, 22 (task pipeline view only), 24, 26, 27, 28. Below the line for MVP: GitHub PR write-automation, the full 14-screen dashboard, Excel, and benchmark productization beyond the evaluation harness. Those stay in the plan for full Specification coverage and are scheduled immediately post-MVP.

7.4 Milestones

Milestone	Phases	Entry	Exit
M0 Capability spike & walking skeleton	0, 1	plan approved	capability floors met; one real task runs REQUIREMENT -> VERIFIED DIFF headless
M1 Repository understanding	2–5	M0 exit	ingest + analyze + graph a real Python repo; golden snapshots stable
M2 Planning	6–9	M1 exit	task -> mapping (evidence) -> impact -> validated plan; validator blocks bad plans
M3 Implement + execute	10–12	M2 exit	real patch + generated tests + secure sandbox execution; escape tests pass
M4 Debug + regression	13–15	M3 exit	bounded repair to green on a seeded bug; regression selection with rationale
M5 Review + score + verify	16–18	M4 exit	review findings, deterministic PCS/CRS, verification verdict; no false-complete on negatives
M6 Git + memory	19–20	M5 exit	Git intelligence answers the four questions; memory retrieval works; PR only after VERIFIED
M7 Product surfaces	21–23	M6 exit	connected pipeline via the API; task pipeline view shows real data; (Excel deferred)
M8 Prove it	24–25	M7 exit	full 18-step E2E green; 16 metrics measured; competitive delta computed; scoring calibrated
M9 Harden	26–28	M8 exit	security suite gating; soak leak-free; 30-point acceptance contract signed

7.5 Kill criteria and decision gates

Gate	When	Metric(s)	Threshold	Action if missed
G0	end of Stage A	localization recall@10; end-to-end verified-fix rate	>= 0.75; >= 0.30	two bounded retrieval/planning redesign rounds; then re-scope to assisted mode or a narrower task class
G1	M2 exit	localization recall@10 on the **held-out** set; plan-validation reject rate on bad plans	>= 0.70; >= 0.95	pause breadth; invest in mapping/planning until recovered
G2	M5 exit	false-complete rate on the labeled set	<= 2%	do not proceed to product surfaces; tighten verification criteria
G3	M8 exit	competitive resolution-rate delta (#15); cost per verified task (#13); replay fidelity (#16)	delta on quality >= 0; cost within `docs/COST_MODEL.md` envelope; fidelity >= 0.9	hold release; address the failing dimension

Critical path and risk: the **capability spike** (Stage A) and **Phase 12 (sandbox)** are the two highest-risk items; the Phase 2 Docker spike de-risks the second early. Phases 9, 10, 14, 21, 22 are the next most demanding.

---

8. Phase 0 — Greenfield Architecture & Planning

Status: COMPLETE — 2026-09-04. All fourteen architecture/strategic documents, twenty ADRs, the 28-item coverage checklist, and the capability-spike harness are delivered (linked below). The capability spike's **mock** run passed (proves the harness); its **live** run against a real provider and the ~30-task benchmark subset remains explicitly **PENDING** — see `docs/CAPABILITY_SPIKE.md` §5.2. Per this plan, Phase 1 (walking skeleton) may proceed; gate G0 must be re-evaluated with live numbers before Phase 2 breadth work accelerates.

Goal. Establish the architecture, the MVP boundary, the data model, the state machine, the agent design, the AI schemas, the sandbox model, the repository model, the scoring algorithms, the positioning and cost model, and the acceptance-test outline — **and prove the core capability with a throwaway spike (Stage A, Section 7.1)** — then immediately begin the walking skeleton. Do not stop at documents.

Depends on. Nothing. This is the first phase.

Deliverables. (■ = produced; see the linked file)

- ■ [`docs/AEGIS_ARCHITECTURE.md`](AEGIS_ARCHITECTURE.md), this document (`docs/AEGIS_IMPLEMENTATION_PLAN.md`), ■ [`docs/TECH_STACK.md`](TECH_STACK.md), ■ [`docs/DATA_MODEL.md`](DATA_MODEL.md), ■ [`docs/SECURITY_MODEL.md`](SECURITY_MODEL.md), ■ [`docs/MVP_DEFINITION.md`](MVP_DEFINITION.md), ■ [`docs/AI_AGENT_DESIGN.md`](AI_AGENT_DESIGN.md), ■ [`docs/METRICS.md`](METRICS.md), ■ [`docs/EXECUTION_MODEL.md`](EXECUTION_MODEL.md), ■ [`docs/REPOSITORY_ANALYSIS.md`](REPOSITORY_ANALYSIS.md).
- ■ [`docs/POSITIONING.md`](POSITIONING.md) (Section 2.1–2.4: wedge, target user, non-goals, competitive baseline), ■ [`docs/COST_MODEL.md`](COST_MODEL.md) (per-task cost envelope + assumptions, Appendix F), ■ [`docs/GOVERNANCE.md`](GOVERNANCE.md) (Section 4.14: replay, audit chain, RBAC, data handling, Trust Report), ■ [`docs/EVAL_HARNESS.md`](EVAL_HARNESS.md) (Section 5.6: benchmark subset, held-out set, reference agents).
- ■ [`docs/DECISIONS/`](DECISIONS/README.md) — one ADR per item in Specification Section 51 (database schema, analysis state machine, job model, agent orchestration model, AI provider abstraction, repository abstraction, code-analysis abstraction, patch representation, artifact storage, sandbox architecture, security policy, repository limits, test execution model, Git strategy, GitHub strategy, memory architecture, metric algorithms, frontend state model) — plus ADRs for model-routing policy and the capability floors (`ADR-0001`..`ADR-0020`).
- ■ [`docs/PHASE0_CHECKLIST.md`](PHASE0_CHECKLIST.md) — coverage of all 28 definitions in Specification Section 46.
- ■ [`docs/CAPABILITY_SPIKE.md`](CAPABILITY_SPIKE.md) + a runnable throwaway harness (`scripts/capability_spike/`) — the mock run is measured and deterministic; the live Stage A result (localization recall@k, end-to-end verified-fix rate on the ~30-task subset) and the go / redesign / re-scope decision are **PENDING** a real provider + the assembled benchmark subset.

Key design decisions. All of Section 4 of this plan, including the cost/latency budget (4.13) and the trust/governance model (4.14). Where the Specification leaves a choice ambiguous, pick the simplest safe option, record the decision in an ADR, and continue.

Implementation steps.

1. Inspect the workspace; confirm greenfield; read the full Specification.

2 Draft the ten Specification documents plus `POSITIONING.md`, `COST_MODEL.md`, `GOVERNANCE.md`, `EVAL_HARNESS.md`.

1 Write one ADR per Section 51 decision (plus model-routing and capability-floor ADRs).

2 Fill the 28-item coverage checklist; resolve every gap.

3 Freeze `scoring-model v1.0.0` constants and the metric formulas; freeze the priced model table used for the cost metric.

1 Assemble the ~30-task capability benchmark subset (Section 5.6) with gold patches/tests.

2 **Run the Stage A capability spike** with throwaway wiring; record results and the go/no-go decision in `docs/CAPABILITY_SPIKE.md`. If below a floor, do up to two bounded retrieval/planning redesign rounds; if still below, invoke gate G0 (Section 7.5).

1 Approve this plan; start Phase 1 (walking skeleton).

Phase-wise testing.

- *Unit:* a `docs` link-checker; a checklist-completeness script; a script that parses the metric and scoring tables so later CI can assert code-vs-doc sync.
- *Integration:* the capability-spike harness runs end-to-end on the benchmark subset and emits a machine-readable results file.
- *Acceptance:* every Section 46 item maps to a document section; every Section 51 decision has an ADR; the scoring section defines signals + normalization + weights + formula + thresholds + version; `CAPABILITY_SPIKE.md` records real measured numbers (not placeholders) and an explicit decision.
- *Regression:* a CI job fails if a later phase changes a scoring constant or the priced model table without bumping the relevant `*_version` and updating `docs/METRICS.md` / `docs/COST_MODEL.md`.

Quality gates (exit criteria). Fourteen documents exist and are internally consistent — ■ met; ADR log complete — ■ met (20/20); 28 definitions covered — ■ met, zero gaps (`PHASE0_CHECKLIST.md`); the priced model table and cost envelope are fixed — ■ met (`pricing-table v1.0.0`, provisional); **the Stage A capability floors are met, or a documented redesign/re-scope path is recorded — ■ open: harness built and passing on mock, live evaluation against G0 is pending a provider key + the benchmark subset**; plan approved — ■ met. Per plan Section 7.1, this open item does not block starting Phase 1, but must close before Phase 2 breadth work accelerates.

Metrics touched. Defines all 16; measures #1 and #5 on the mock wiring-proof set only (not yet a capability measurement — see `CAPABILITY_SPIKE.md`); establishes the priced model table for #13.

Risks & mitigations. Analysis paralysis -> timebox; "simplest safe option + ADR + continue". Over-specification -> mark provisional items for Phase 25 calibration. Spike under-performs -> bounded redesign then honest re-scope rather than proceeding on a weak core.

Effort. L (was M; the capability spike adds real work — and de-risks everything after it).

---

9. Phase 1 — Walking Skeleton

Status: COMPLETE — 2026-09-04. `backend/aegis/` implements the full reduced pipeline; 47 tests pass (1 Docker-gated test auto-skips — no Docker daemon in this environment). The CLI reaches **VERIFIED** end-to-end on the seeded acceptance task via `--sandbox fake` (proving the pipeline logic); with the real `DockerSandboxRunner` (the default), it correctly and cleanly returns `PARTIALLY_SUPPORTED{reason: docker`

`unavailable}` — the honest, designed behaviour in an environment without Docker, never a host-execution fallback. The unfixable fixture (`test-repositories/fixtures/unfixable`) ends `SAFE_STOP` cleanly after exactly its 2-attempt repair budget. **Open item:** the real Docker happy path (gate: `test_d_real_docker_...` in `backend/tests/e2e/test_skeleton_pipeline.py`) has not been run anywhere Docker is installed.

Goal. Get **one real engineering task** from requirement to a verified printed diff through a single connected pipeline, headless, as fast as possible — proving the integration end-to-end before any breadth is built. This is Stage B of the delivery strategy (Section 7.2).

Depends on. Phase 0. Builds deliberately reduced "just enough" slices of Phases 2, 3, 4, 7, 9, 10, 12, 13, 14, 18 — not their full versions.

Deliverables.

- A minimal `backend/app` that runs a single task synchronously (no job queue, no worker, no DB

  beyond a scratch SQLite for artifacts) via one CLI entry point `python -m aegis.skeleton run <repo> <task.md>`.
- Reduced modules: `repository/ingest` (local path only), `analysis/python_ast` (symbols +

  imports only), `analysis/mapping` (lexical FTS + import-graph proximity, no embeddings), `agents/planning` (single frontier-model call, schema-validated), `implementation/editor` (anchored edits + unified diff), `sandbox/runner` (Docker, `--network none`, the core resource caps), `debugging/repair_loop` (max 2 iterations), `verification/agent` (three mandatory criteria: issue tests pass, no unplanned files, patch re-applies).
- The `TrustReport` v0 (Section 4.14): evidence trace + raw signals + verification verdict, as

  JSON.
- `backend/tests/e2e/test_skeleton_pipeline.py`.

Key design decisions. Ruthless scope: no dashboard, no engineering memory, no GitHub, no Excel, no calibrated scoring (emit raw signals), no regression selection (run the repo's full suite), no code-review AI pass (static checks only). Everything uses the `MockProvider` in tests and a real `frontier` provider when `RUN_LIVE_AI=1`. The skeleton's `EngineeringPlan`, `ImplementationResult`, and `VerificationResult` schemas are the **real** schemas from Section 4.6 — reduced implementations, not reduced contracts — so later phases thicken behind stable interfaces.

Implementation steps.

1. CLI entry point + synchronous pipeline runner threading typed outputs stage to stage.
2. Reduced ingest + analysis + lexical/graph mapping.
3. Single-call planning against the real `EngineeringPlan` schema + schema guard.
4. Anchored editor + unified-diff generation on a copy-on-write workspace.
5. Docker sandbox run of the repo's test command + result parse.
6. Two-iteration repair loop reusing the editor and sandbox.
7. Verification of the three mandatory criteria + `TrustReport` v0 emission.
8. Wire the E2E test on one real task from the acceptance-repo seed (Phase 3 minimal version).

Phase-wise testing.

- *Unit:* the pipeline runner passes each stage's output object unmodified to the next (typed, no

  re-derivation); the reduced mapper returns evidence-bearing candidates; the two-iteration cap is enforced; the `TrustReport` v0 serializes with all required fields.
- *Integration:* on a small fixture repo with a known one-line bug, the skeleton produces a diff,

runs tests in the sandbox, and reaches a verdict; a fixture with no fix available ends cleanly (no crash, `NOT_VERIFIED` + partial `TrustReport`).

- *Acceptance:* one real task on the minimal acceptance repo goes **REQUIREMENT -> VERIFIED DIFF**

  headless via the CLI; the printed diff applies to a clean checkout; the `TrustReport` names the file, the change reason, and the passing tests.

- *Regression:* `test_skeleton_pipeline.py` becomes the **top-level regression anchor** run in CI

  on every change until Phase 24's controlled-repo E2E supersedes it; a determinism test replays the mock-provider task and asserts an identical diff (Section 5.8).

Quality gates (exit criteria). One real task reaches a verified diff headless; the pipeline is connected (stage-input == prior stage-output, asserted); the sandbox runs with `--network none` and the core caps; the run stays within a provisional cost/latency band; the E2E is green in CI.

Metrics touched. #1, #4, #5, #12 (provisional, single task); #13/#14 provisional band; #16 (replay).

Risks & mitigations. Temptation to over-build the skeleton -> the scope list above is a contract; anything not on it waits for its phase. Skeleton code becoming load-bearing -> reduced implementations sit behind the real schemas and are replaced, not extended, in later phases.

Effort. L.

---

10. Phase 2 — Project Foundation

Goal. A runnable, tested skeleton: repository layout, FastAPI app, configuration, structured logging with secret redaction, database + migrations, typed error envelope, Docker Compose, CI, and the base test harness.

Depends on. Phase 0.

Deliverables.

- `backend/app/main.py` (app factory), `core/config.py`, `core/logging.py`, `core/errors.py`,

  `core/security.py`, `core/limits.py`.

- `db/session.py`, `models/base.py`, first Alembic migration (`Job`, `AuditLog`).
- `GET /healthz`, `GET /version`.
- `frontend/` scaffold: Vite + React + TS + router + query client + typed API client stub.
- `docker/`, `docker-compose.yml` (api, worker placeholder, frontend, optional postgres).
- `.github/workflows/ci.yml`, `pyproject.toml` with pinned deps + lockfile, `pre-commit` config,

  `CONTRIBUTING.md`, run/dev docs.

Key design decisions. `pydantic-settings` config validated at startup; JSON logging with a correlation/task id and a redaction filter for known secret patterns; typed error envelope `{code, message, details, evidence?}`; SQLAlchemy 2.0; repository pattern for data access; UTC timestamps; sortable UUID ids; sync engine acceptable for dev, async optional and documented.

Implementation steps.

1. Create the directory tree from Section 4.2.
2. Pin dependencies; generate a lockfile.
3. App factory + `/healthz` + `/version` (build metadata).
4. Config module + fail-fast validation.
5. Logging + redaction filter.

6 DB session + `Base` + first migration.

7 Error handlers + request-id middleware.

8 Frontend scaffold + API client + env wiring.

9 Docker Compose for the whole dev stack.

10 CI: ruff + black + mypy + pytest + coverage gate; frontend build + ESLint + `tsc`.

11 `pre-commit`; developer docs.

12 Sandbox spike (throwaway): confirm Docker CPU/memory/pids limits work on the CI runner; record

findings for Phase 12.

Phase-wise testing.

- *Unit:* config parsing (valid / missing / malformed env -> clear error); redaction filter masks

token/key/password patterns; error-envelope serialization; id generator uniqueness and sort order.

- *Integration:* app boots under TestClient; `/healthz` 200; `/version` returns build metadata;

Alembic upgrade then downgrade on a fresh SQLite file; 404 and 422 produce the typed envelope; CORS headers present.

- *Acceptance:* `docker compose up` brings API + frontend to healthy; CI is green on a clean

checkout; a coverage report is produced.

- *Regression:* establishes the baseline suite and the coverage floor that every later phase must

keep green.

Quality gates. App starts; migrations reversible; CI green; lint/type clean; a test proves no secret reaches the logs; frontend builds.

Metrics touched. None (infrastructure).

Risks & mitigations. Dependency drift -> pin + lockfile + Renovate later. Async DB complexity -> start sync, document the async path.

Effort. M.

---

11. Phase 3 — Repository Ingestion

Goal. Ingest a Git repository (local path or GitHub URL) into an isolated, read-only workspace snapshot; persist metadata; enforce size limits; handle every ingestion failure mode as a structured state.

Depends on. Phase 2.

Deliverables.

- `repository/ingest.py`, `repository/workspace.py`, `repository/git_client.py` (GitPython wrapper),

`repository/url_validator.py`, `repository/limits.py`.

- Models: `Repository`, `RepositorySnapshot`, `RepositoryFile`, `Artifact`.
- Schemas: `RepositoryRef`, `IngestRequest`, `IngestResult`.
- API: `POST /repositories`, `GET /repositories/{id}`, `POST /repositories/{id}/snapshots`.
- Job type `INGEST`; state `INGESTING`.
- A minimal `test-repositories/aegis-acceptance/` so later phases have a real target.

Key design decisions. Clone with configurable depth; disable hooks (`core.hooksPath=/dev/null`), submodule execution, and credential prompts; workspace under `artifacts/workspaces/<snapshot_id>` with restricted permissions; manifest = file list + sizes + SHA-256 + language (extension + shebang); URL allowlist (`https` + `github.com` + configurable hosts), reject `ssh`/`file`/`git` schemes and private/loopback IPs (SSRF); exceeding a limit -> `PARTIALLY_SUPPORTED{reason}`; never run repository code here.

Implementation steps.

1. URL validator + SSRF guard.
2. Git client: clone / fetch / checkout with timeouts and hooks disabled.
3. Workspace manager: create / lock / reset / cleanup.
4. Manifest builder: hashing + language detection.
5. Limit enforcement with structured partial-support result.
6. Persistence: repository / snapshot / file rows.
7. API + job wiring.
8. Failure mapping: invalid / private / inaccessible / rate-limited / network -> distinct states.

Phase-wise testing.

- *Unit:* URL validator (accept `https://github.com/...`; reject `file://`, `git@...`,

`http://169.254.169.254`, `http://localhost`, punycode homographs); limit checks; language detection table; manifest hashing determinism.

- *Integration:* ingest a local fixture repo -> snapshot + file rows correct; ingest a bare-repo

fixture simulating GitHub -> same; re-ingesting the same commit is idempotent (dedup by snapshot hash); workspace is read-only; cleanup removes the workspace; an oversized fixture returns `PARTIALLY_SUPPORTED`.

- *Acceptance:* `POST /repositories` then `POST /snapshots` on the acceptance repo yields the

expected file count with Python as the dominant language; metadata is retrievable via the API.

- *Regression:* full suite green; a subprocess spy asserts no host process other than `git`/`docker`

is spawned during ingestion.

Quality gates. Real repository loaded; metadata stored; files accessible via API; invalid repositories handled; SSRF tests pass; workspace isolated and reversible.

Metrics touched. None directly (enables all later ones).

Risks & mitigations. GitHub rate limits in CI -> use local bare-repo fixtures. Large repos slow -> depth + limits.

Effort. M.

---

12. Phase 4 — Repository Intelligence (Python analysis)

Goal. Parse Python sources into structured facts: modules, imports, functions, classes, methods, signatures, docstrings, decorators, entry points, configuration, test files, package manager, build backend, and runtime/test commands — with `UNKNOWN` where undeterminable.

Depends on. Phase 3.

Deliverables.

- `analysis/python_ast.py`, `analysis/symbols.py`, `analysis/imports.py`, `analysis/project_meta.py`

(pyproject / setup.cfg / requirements / tox / pytest.ini), `analysis/entrypoints.py`, `analysis/testdetect.py`.

- Models: `RepositorySymbol`, `Dependency` (kind = import / package), `RepositoryAnalysis`.
- Schema: `RepositoryAnalysis` / `RepositoryContext`.
- API: `GET /analysis/{snapshot_id}`; state `ANALYZING`.

Key design decisions. Standard-library `ast` only (never import or exec target code); `tree-sitter` behind a flag for files with syntax errors and for future languages; stable symbol id `"{relpath}::{qualname}"` with line ranges; detect test framework from config and markers; detect package manager (pip / poetry / uv / pipenv) and build backend; record `UNKNOWN` rather than guess; an unparseable file is recorded with its error and analysis continues.

Implementation steps.

1. File walker honouring `.gitignore`, skipping vendored/generated trees.
2. AST visitor -> symbols, signatures, decorators, and callee *names* (no resolution yet).
3. Import extractor classifying stdlib / third-party / local using the manifest + a stdlib list.
4. Project-metadata parsers for poetry, PEP 621, and requirements files.
5. Entry-point detection (`__main__`, console_scripts, ASGI/WSGI app objects, CLI).
6. Test-infrastructure detection (framework + command).
7. Persist facts and assemble `RepositoryContext`.
8. Partial-failure handling for unparseable files.

Phase-wise testing.

- *Unit:* AST visitor on fixtures (nested classes, async defs, decorators, comprehensions,

conditional imports); import classifier; stdlib detection; metadata parsers for each layout; syntax-error file captured, not fatal.

- *Integration:* analyze the acceptance repo -> symbol counts, import edges, detected framework and

command match a golden JSON snapshot; performance within budget for N files.

- *Acceptance:* `RepositoryContext` contains entry points, the test command, and the package

manager for the acceptance repo; unknowns are explicit `UNKNOWN`.

- *Regression:* golden-file diffs on three fixture repos; determinism across runs.

Quality gates. Files parsed; symbols extracted; dependencies detected; results persisted; unparseable files handled; golden snapshots stable.

Metrics touched. Feeds #1 (mapping) and #2 (alignment) later.

Risks & mitigations. Exotic or newer syntax -> `tree-sitter` fallback. Monorepo noise -> gitignore + vendor skiplist.

Effort. L.

---

13. Phase 5 — Code Graph & Dependency Analysis

Goal. Build a knowledge graph (NetworkX in memory, adjacency in the DB) with nodes (repo / file / module / class / function / test / dependency / commit / issue / patch) and edges (`IMPORTS`, `CALLS`, `DEFINES`, `TESTS`, `MODIFIES`, `DEPENDS_ON`, `CHANGED_BY`, `RELATED_TO`, `FIXED_BY`, `AFFECTS`). Provide queries: neighbours, callers/callees, test-to-code, k-hop impact set, shortest path, centrality.

Depends on. Phase 4 (Git edges enriched in Phase 19).

Deliverables.

- `analysis/graph/builder.py`, `graph/store.py`, `graph/queries.py`, `graph/centrality.py`.
- Adjacency tables + a serialized-graph Artifact.
- Schema: `CodeGraphSummary`.
- API: `GET /analysis/{snapshot}/graph`, `.../graph/subgraph`, `.../graph/node/{id}`.

Key design decisions. Call edges resolved by name + import table + same-module scope, each labelled `RESOLVED` / `HEURISTIC` / `UNRESOLVED`; dynamic dispatch -> `UNRESOLVED` (never asserted as `FACT`); persist edges for querying and keep NetworkX for algorithms; degree + (approximate for large graphs) betweenness centrality feed the risk score; test-to-code links from test-file imports and `test_<module>` naming.

Implementation steps.

1. Node/edge id scheme.
2. Builder from symbols + imports.
3. Call-edge resolver with a confidence label.
4. Test linkage.
5. Persistence + reload equality.
6. Query API (callers, callees, k-hop impact, path).
7. Centrality with caching.
8. Subgraph export for visualization.

Phase-wise testing.

- *Unit:* resolver on fixtures (local call, imported call, aliased import, method call, unresolved dynamic); k-hop impact set; centrality on a hand-computed small graph.
- *Integration:* the acceptance-repo graph matches a golden edge set within tolerance; a graph reloaded from the DB equals the in-memory graph; query latency within budget.
- *Acceptance:* "callers of `calculate_total`" returns `checkout` and `order_service` (the Specification's worked example); unresolved edges are labelled.
- *Regression:* golden graph snapshots; stable node/edge ordering for determinism.

Quality gates. Graph built, persisted, and reloadable; caller/callee queries correct on fixtures; centrality reproducible; unresolved edges labelled.

Metrics touched. Feeds #1, #6, and the CRS centrality signal.

Risks & mitigations. Call-graph imprecision -> explicit confidence + never `FACT`. Graph size -> approximate algorithms + limits + `PARTIALLY_SUPPORTED`.

Effort. L.

---

14. Phase 6 — Task / Issue Ingestion (Normalization)

Goal. Accept an issue / bug / feature / requirement (free text or a GitHub issue reference) and produce a normalized `Task` with type, title, description, acceptance hints, any user-supplied files/symbols, constraints, and priority; create `Task` + `TaskStep` + `Job`; move `PENDING -> QUEUED`.

Depends on. Phase 2 (Phase 3 and 19 for GitHub issue import, optional).

Deliverables.

- `services/tasks.py`; schemas `TaskCreate`, `Task`, `IssueAnalysisInput`; models `Task`, `TaskStep`, `Issue`.
- API: `POST /tasks`, `GET /tasks/{id}`, `POST /tasks/{id}/run`, `POST /tasks/{id}/cancel`, `GET /tasks/{id}/timeline`.

Key design decisions. Treat issue text as untrusted: strip control characters, cap length, never interpolate it into system prompts (structured fields + guarding only); `task_type` enum `{BUG, FEATURE, REFACTOR, REQUIREMENT, QUESTION}` inferred by deterministic rules with optional AI enrichment (produced as `IssueAnalysis` in Phase 7); idempotency key `hash(repo, normalized_text)` with duplicate detection; cancellation is a cooperative flag checked between stages.

Implementation steps.

1. `Task` / `Issue` / `TaskStep` models + migration.
2. Create / list / get services with pagination and filtering.
3. Normalization rules + validation.
4. Job creation + state transitions + idempotency + duplicate detection.
5. `run` / `cancel` endpoints.
6. Timeline aggregation (steps + job events).
7. Optional GitHub issue import adapter behind config.

Phase-wise testing.

- *Unit:* normalization (whitespace, markdown, oversized input truncation with provenance, injection strings neutralized); `task_type` inference rules; idempotency hash; duplicate detection.
- *Integration:* `POST /tasks` persists and returns; `run` enqueues a job and sets `QUEUED`; `cancel` sets `CANCELLED` and halts progression; the timeline reflects steps.
- *Acceptance:* submitting the Specification's example task text stores a `Task` with a sensible type and the description intact.
- *Regression:* OpenAPI schema snapshot; prior suites green.

Quality gates. Real task submitted and retrievable; job created; states explicit; cancellation works; inputs sanitized.

Metrics touched. Feeds #12 (completion rate).

Risks & mitigations. Prompt injection via issue text -> never concatenate raw text into prompts; structured fields only; documented in the security model.

Effort. S–M.

---

15. Phase 7 — Issue -> Code Mapping

Goal. Produce an evidence-backed mapping `Issue -> {files, symbols, tests, dependencies, confidence}` by combining lexical search, semantic retrieval, symbol relationships, the code graph, Git history (when present), test relationships, repository structure, and engineering memory (when present).

Depends on. Phases 4, 5, 6. Git (18) and memory (19) are optional enrichers wired via hooks now.

Deliverables.

- `analysis/mapping/lexical.py` (SQLite FTS5 over code + docstrings + symbols),

  `mapping/semantic.py` (embeddings via `AIProvider.embed` or a local model, optional), `mapping/fuse.py` (reciprocal-rank fusion), `mapping/evidence.py`.
- Schema: `IssueCodeMapping` (per candidate: typed evidence list, score, confidence,

  `FACT`/`INFERENCE` labels).
- API: `GET /tasks/{id}/mapping`, `POST /analysis/map`.

Key design decisions. Each retriever emits candidates with concrete evidence (lexical hit line, symbol match, graph proximity, historical commit, test-coverage link). Reciprocal-rank fusion with documented weights (`mapping-model v1.0.0`). Confidence is a calibrated function of cross-retriever agreement and the top-score margin. Hard rule: **no candidate without at least one concrete evidence item.** Semantic retrieval is optional; without an embeddings provider the system degrades to lexical + graph and reports reduced confidence. Deterministic given a fixed index and model.

Implementation steps.

1. Build the FTS index (symbols + docstrings + code).
2. Optional embedding index chunked by symbol, behind the provider abstraction with a local

   fallback flag.
1. Per-retriever candidate generation with evidence.
2. Rank fusion + configurable weights.
3. Confidence calibration (heuristic now, calibrated in Phase 25).
4. Mapping schema + persistence + API.
5. Memory / Git enrichment hooks (no-ops when absent).

Phase-wise testing.

- *Unit:* FTS query building and ranking; RRF math; evidence object shape; confidence monotonicity

  (more agreement never lowers confidence); the no-embeddings degraded path.
- *Integration:* on the acceptance repo, the mapping for "discount exceeds maximum" returns the

  invoice module and `calculate_total` in the top-k with evidence (golden expectation, tolerant of ordering).
- *Acceptance:* every returned candidate carries evidence; a confidence value is present; `UNKNOWN`

  when nothing passes threshold.
- *Regression:* metric #1 on a curated mini-set holds at or above target (e.g. recall@5 >= 0.8);

  ranked-list snapshots stable.

Quality gates. Relevant files retrieved; evidence attached; confidence calculated; deterministic; no evidence-free candidates.

Metrics touched. #1 (primary).

Risks & mitigations. Embeddings availability/cost -> optional + fallback. Overfitting to fixtures -> separate calibration and test sets.

Effort. L.

---

16. Phase 8 — Impact Analysis

Goal. From the mapping and the graph, compute affected files and functions, direct callers, indirect dependencies, related tests, public APIs, configuration and database references (where detectable), and likely regression areas -> an `ImpactAnalysis` report plus a bundle of risk signals.

Depends on. Phases 5, 7.

Deliverables.

- `analysis/impact.py`.
- Schema: `ImpactAnalysis` (changed set, blast radius by hop, callers, related tests, `public_api_touched`, `config_refs`, `db_refs`, `regression_areas`, `risk_signal_bundle`).
- API: `GET /tasks/{id}/impact`.

Key design decisions. Blast radius via reverse-graph BFS to a configurable depth; public API = symbols in `__all__`, package top level, or FastAPI routes; config references = uses of settings / env keys; database references = ORM model references and SQL string literals (best effort, labelled `INFERENCE`). Output is both a human-readable report matching the Specification's example format and a machine bundle feeding the CRS in Phase 17.

Implementation steps.

1. Reverse-dependency and caller extraction.
2. Related-test selection (graph `TESTS` edges + heuristics).
3. Public-API detection.
4. Config / DB reference scan.
5. Regression-area ranking (`centrality * coverage_gap`).
6. Report assembly.
7. Persistence + API.

Phase-wise testing.

- *Unit:* reverse-BFS depth control; public-API detector on fixtures; config/env scanner; DB-ref heuristic labelled `INFERENCE`.
- *Integration:* a change to `invoice.py` on the acceptance repo yields a report listing `calculate_total`, callers `checkout.py` / `order_service.py`, tests `test_invoice.py` / `test_checkout.py`, and `Risk: MEDIUM` with a reason string — matching the Specification's example shape.
- *Acceptance:* the report is evidence-backed, machine-readable, and persisted; no `FACT` claims for heuristic items.
- *Regression:* golden impact snapshots on fixtures.

Quality gates. Affected files / functions / callers / tests produced; report matches the documented schema and example; persisted.

Metrics touched. Feeds CRS and #2.

Risks & mitigations. Over-broad blast radius -> depth cap + ranking. False DB/config detection -> label + low confidence.

Effort. M.

---

17. Phase 9 — Engineering Planning + Plan Validation

Goal. The Planning Agent produces a machine-readable `EngineeringPlan` (problem interpretation, assumptions, files to inspect, files to modify, symbols to modify, dependencies, implementation steps, test strategy, expected behaviour, regression risks, rollback strategy, confidence). The Validator checks schema, feasibility, scope, and evidence **before** implementation.

Depends on. Phases 7, 8. This phase delivers the first real use of the AI provider abstraction (interface defined in Phase 0, stub in Phase 2).

Deliverables.

- `agents/planning.py`; `ai/provider.py`, `ai/claude.py`, `ai/openai.py`, `ai/local.py`, `ai/mock.py`, `ai/prompt.py`, `ai/schema_guard.py`.
- Schemas: `EngineeringPlan`, `PlanValidation`.
- API: `GET /tasks/{id}/plan`, `POST /tasks/{id}/plan/validate`; states `PLANNING`, `PLAN_VALIDATION`.

Key design decisions. Provider abstraction: templated structured prompts, JSON-schema-constrained output, one repair round on invalid output then clean failure, retry with backoff, per-call timeout, token budgeting, redacted logging. Every AI output is schema-validated before its consumer runs. The plan must reference mapping / impact evidence ids. Validator verdict `{APPROVED, REVISE(reasons), REJECTED}` from: (a) JSON schema, (b) referenced files exist in the snapshot, (c) modify-set is a subset of `mapping ∪ impact ∪ explicit allowlist`, (d) every step has a test intent, (e) a rollback strategy is present, (f) assumptions and confidence are non-empty. A deterministic rule-based fallback plan (skeleton from the impact set, flagged `LOW` confidence) is produced when no AI provider is configured.

Implementation steps.

1. `AIProvider` interface + config-driven selection + at least one real provider + mock/local.
2. Prompt templates with a schema and few-shot examples.
3. Schema guard + retry/repair.
4. Planning-agent orchestration (inputs: `RepositoryContext` slice, `IssueCodeMapping`, `ImpactAnalysis`).

1. `EngineeringPlan` schema + persistence.
2. `PlanValidation` rules engine.
3. API + state transitions.
4. Request logging + AI metrics.

Phase-wise testing.

- *Unit:* schema guard (valid, missing field, wrong enum, extra keys); retry/repair with a stub provider returning bad-then-good; provider timeout and error handling; each validator rule (missing file, scope escape, step without test, no rollback); the deterministic fallback plan.
- *Integration:* with `MockProvider` returning a canned schema-valid plan, the full `PLANNING -> PLAN_VALIDATION` transition; a `@pytest.mark.live_ai` smoke test with a real provider.
- *Acceptance:* the acceptance-repo task yields a plan naming the invoice module and `calculate_total`, a boundary-validation step, a test strategy, and a rollback; the validator `APPROVES` it; a deliberately scope-escaping plan is `REJECTED`.
- *Regression:* schema contract snapshots; a test asserts no untrusted text or secret appears in

logged prompts; metric #2 harness stub.

Quality gates. Valid schema; affected files listed; implementation steps present; test strategy present; the validator blocks bad plans; AI output schema-validated; the pipeline runs both without a real provider (fallback) and with one.

Metrics touched. #2 (alignment).

Risks & mitigations. Provider variability -> schema constraint + repair + fallback. CI cost / flakiness -> mock by default, real behind a marker.

Effort. L.

---

18. Phase 10 — Real Patch Generation (Autonomous Implementation)

Goal. The Implementation Agent applies the approved plan to a writable copy of the workspace, producing a real unified diff. It enforces minimal scope, keeps every change reversible, and supports file creation, modification, insertion, replacement, import edits, configuration changes, and test additions.

Depends on. Phases 3 (workspace), 9.

Deliverables.

- `implementation/agent.py`, `implementation/editor.py` (safe anchored edits),

  `implementation/patcher.py` (unified-diff generate/apply), `implementation/workspace_rw.py`, `implementation/scope_tracker.py`.
- Schema: `ImplementationResult` (per step: file, hunk, `plan_step_id`, rationale, evidence).
- Models: `Implementation`, `Patch`.
- API: `GET /tasks/{id}/changes`; state `IMPLEMENTING`.

Key design decisions. Copy-on-write workspace from the snapshot with a baseline commit. The AI proposes edits as structured operations (path + anchor + old/new, or full content for new files) — never a blind overwrite. The editor validates anchors and rejects ambiguous or missing matches. Diffs are generated from the throwaway workspace. The scope tracker compares touched files against the plan allowlist live; an out-of-scope write is blocked (hard stop unless policy overrides). The original snapshot is never mutated.

Implementation steps.

1. RW workspace clone + baseline commit.
2. Structured edit-operation schema + prompt.
3. Anchored editor with ambiguity/safety checks.
4. Apply operations; bounded regeneration on conflict.
5. Diff generation + patch Artifact.
6. Traceability links (plan step <-> hunk).
7. Scope tracker + violation events.
8. API + state.

Phase-wise testing.

- *Unit:* editor anchor matching (unique / ambiguous / missing); new-file creation; idempotent

  import insertion; diff generate-then-reapply equality; scope tracker flags an extra file.
- *Integration:* `MockProvider` returns edit operations for the acceptance task -> the patch applies

to the RW workspace, the original snapshot is unchanged, and the diff touches only the invoice module; an out-of-scope operation is blocked and recorded.

- *Acceptance:* a real diff is generated; the patch applies cleanly; the original is preserved; each

  hunk traces to a plan step.

- *Regression:* patch apply/rollback round-trip on fixtures; a filesystem spy asserts no write

  outside the workspace.

Quality gates. Real diff generated; patch applies; original preserved; traceability recorded; scope tracked.

Metrics touched. #2, #7.

Risks & mitigations. Fragile anchors -> require context lines and fail loudly. AI over-editing -> scope guard + minimal-diff prompt + the PCS size penalty.

Effort. L.

---

19. Phase 11 — Test Generation

Goal. The Testing Agent generates tests for the changed behaviour, considering the existing framework, nearby tests, changed functions, public behaviour, edge / negative / boundary / regression / issue-specific cases. Tests are written into the RW workspace and tracked as `Generated / Executed / Passed / Failed / Skipped / Invalid`.

Depends on. Phases 4 (framework detection), 10.

Deliverables.

- `testing/generator.py`, `testing/selector.py`, `testing/catalog.py`.
- Schema: `TestGeneration` (per test: name, path, `target_symbol`, kind, rationale, evidence,

  status).

- Model: `TestCase`.
- API: `GET /tasks/{id}/tests`.

Key design decisions. Mirror the existing framework and style (pytest fixtures, naming, file placement). Generation is AI-driven with a schema; a test that does not parse or collect is marked `INVALID` (full collection happens in the Phase 12 sandbox). De-duplicate against existing tests. Policy: at least one boundary case and one negative case per changed public function. Never modify unrelated existing tests; modifying an existing test requires an explicit plan justification.

Implementation steps.

1. Framework/style profiler from Phase 4 data.
2. Case-matrix builder (edge / negative / boundary / regression / issue).
3. AI test synthesis + schema.
4. Static validity check (parse; full collect deferred to Phase 12).
5. Catalog + status tracking.
6. Targeted-set selection for execution.
7. API.

Phase-wise testing.

- *Unit:* case-matrix generation for a sample signature; the style profiler; de-duplication;

invalid-test detection (syntax error -> `INVALID`).

- *Integration:* `MockProvider` -> generated test files land in the RW workspace and collect

  locally; catalog statuses are `GENERATED`; the targeted set is computed.

- *Acceptance:* for the acceptance task, at least one boundary test for `discount == max` and

  `discount > max` is created and references the issue.

- *Regression:* metric #3 harness at or above target on fixtures; a test asserts no edits to

  unrelated existing tests.

Quality gates. Tests generated with rationale and evidence; statuses tracked; invalid tests flagged; no unjustified edits to existing tests.

Metrics touched. #3 (primary), #4.

Risks & mitigations. Uncompilable AI tests -> static gate + one repair round. Trivial / no-op tests -> assertion-presence check + mutation check in Phase 25.

Effort. M–L.

---

20. Phase 12 — Secure Execution (Docker Sandbox)

Goal. Execute the repository and the generated tests inside an isolated Docker sandbox with CPU / memory / pids / wall-clock limits, no network by default, no secret exposure, workspace-only filesystem, and full logging. Return a structured `TestExecution`.

Depends on. Phases 2 (Docker + the Phase 2 spike), 11. Highest-risk phase — front-loaded spike in Phase 2.

Deliverables.

- `sandbox/runner.py`, `sandbox/docker_backend.py`, `sandbox/policy.py`,

  `sandbox/resource_limits.py`, `sandbox/result_parser.py`.

- `docker/sandbox.Dockerfile` (base pinned by digest, non-root, minimal).
- Schema: `TestExecution` (command, exit code, durations, per-test results, stdout/stderr Artifact

  refs, resource usage).

- Model: `TestExecution`.
- API: `GET /tasks/{id}/executions`, `GET /executions/{id}`; state `EXECUTING_TESTS`.
- `docs/EXECUTION_MODEL.md` first draft (finalized in Phase 27).

Key design decisions. Full control matrix from Section 4.8: `--network none` by default, `--read-only` rootfs + `tmpfs` scratch + workspace bind `rw` only, `--cap-drop ALL`, `--security-opt no-new-privileges`, `--pids-limit`, `--memory` / `--cpus`, `--ulimit nofile/nproc`, non-root UID, env scrubbed to an allowlist, image pinned by digest, no Docker socket, wall-clock timeout with kill, container and volume always removed. Dependency install is a guarded opt-in pre-step (host-allowlisted, hash-pinned) that briefly enables network. Test command comes from Phase 4 (overridable). Results are parsed from `pytest` JUnit-XML / JSON into per-test statuses. If Docker is unavailable -> `PARTIALLY_SUPPORTED` with a documented reason (no host fallback).

Implementation steps.

1. Sandbox image + CI build.
2. Policy + resource-limit config.
3. Docker backend: create / start / exec / collect / kill / remove with timeouts.

4 Env scrubbing + secret isolation.

5 Guarded dependency-install step.

6 Test-command resolution.

7 Result parser -> per-test statuses.

8 Execution logging + Artifact storage.

9 State + failure states (timeout, OOM, infra).

Phase-wise testing.

- *Unit:* policy builder (asserts `network none`, `cap-drop ALL`, `pids-limit`, `memory`, `no-new-privileges`, `read-only` are all present); env scrubber; result parser on sample JUnit / JSON (pass / fail / error / skip); timeout-kill logic; Docker-unavailable -> `PARTIALLY_SUPPORTED`.
- *Integration (`@pytest.mark.docker`):* a benign fixture's passing suite parses correctly; hostile fixtures — (a) a test that opens a network connection is blocked, (b) a fork-bombing test is stopped by `pids-limit`, (c) a memory hog triggers a handled OOM, (d) a test that writes outside the workspace is denied, (e) a test that reads a seeded secret env var finds nothing — and the container is always removed.
- *Acceptance:* the acceptance-repo tests plus the generated tests execute in the sandbox; results are structured; a host subprocess spy shows only `docker` was invoked.
- *Regression:* the security test suite is a hard gate; resource-limit regression; no leftover containers or volumes after the suite.

Quality gates. Tests actually execute in the sandbox; results captured and parsed; network is off by default (proven); escape and exhaustion tests pass; no secrets inside the sandbox; deterministic cleanup.

Metrics touched. #4, and enables #5, #6, #10.

Risks & mitigations. Docker absent in some environments -> `PARTIALLY_SUPPORTED` + clear messaging. CI Docker performance -> small images + layer cache. Flaky network-block test -> assert the config, not only the runtime behaviour.

Effort. XL.

---

21. Phase 13 — Failure Investigation

Goal. On failing tests, collect structured failure data — stack traces, failing tests, changed files, relevant code slices, recent changes, related tests, dependency context — into a `FailureAnalysis` that separates `FACT` from `INFERENCE` and never invents a cause.

Depends on. Phases 5, 10, 12.

Deliverables.

- `debugging/investigate.py`.
- Schema: `FailureAnalysis` + `FailureRecord`.
- Models: `Failure`, `Investigation`.
- API: `GET /tasks/{id}/failures`; state `INVESTIGATING`.

Key design decisions. Parse tracebacks into frames; map frames to symbols/files via the graph; gather the diff hunks touching those frames, related tests, and recent commits for those files (when Git intelligence is present). Classify the failure type deterministically from the execution output (assertion, exception/error, collection error,

timeout, import error, environment). Output is data plus candidate signals only — no cause is asserted.

Implementation steps.

1. Traceback parser (pytest and unittest formats).
2. Frame -> symbol resolver.
3. Evidence bundler (code slices, diff hunks, related tests, history).
4. Failure-type classifier.
5. Schema + persistence.
6. API + state.

Phase-wise testing.

- *Unit:* traceback parser on varied samples (nested frames, chained exceptions, import/syntax errors, timeouts); frame resolver; classifier mapping; evidence-bundle shape.
- *Integration:* injecting a bug into a fixture then running -> the investigation identifies the correct failing test, file, frame, and code slice; a chained exception is handled.
- *Acceptance:* the acceptance repo with seeded failing behaviour -> `FailureAnalysis` identifies the failing assertion and the implicated function with evidence, and fabricates no cause.
- *Regression:* parser golden tests; `FACT` vs `INFERENCE` labelling verified.

Quality gates. Failures analysed; evidence attached; failure type classified deterministically; no invented causes.

Metrics touched. Feeds #5, #11.

Risks & mitigations. Unusual traceback formats -> raw capture + `UNKNOWN`. Framework variety -> start with pytest, document others.

Effort. M.

---

22. Phase 14 — Autonomous Debugging & Repair (Bounded Loop)

Goal. The Debugging Agent produces an evidence-linked `RootCauseAnalysis` and ranked repair hypotheses (`FACT` / `INFERENCE` / `HYPOTHESIS`), applies a candidate fix, re-runs targeted then regression tests, reviews, and iterates up to a bounded number of times. Every attempt is recorded. If a repair makes things worse, it auto-reverts. If it cannot confidently repair, it **stops safely** with evidence.

Depends on. Phases 10, 11, 12, 13, and a basic version of 15.

Deliverables.

- `debugging/repair_loop.py`, `debugging/rca.py`, `debugging/hypotheses.py`, `debugging/guard.py`.
- Schemas: `RootCauseAnalysis`, `RepairProposal`.
- Model: `RepairAttempt`.
- API: `GET /tasks/{id}/repairs`; state `REPAIRING`.

Key design decisions. Iteration budget (config, default 4) plus a wall-clock budget plus a no-progress detector (the same failure signature twice aborts the loop). Each attempt: RCA (evidence required) -> ranked hypothesis -> edit operations (reusing the Phase 10 editor + scope guard) -> sandbox targeted tests -> if improved, run a regression subset -> snapshot the candidate. The best candidate is kept, ordered by `(failing_count, regression_failures, diff_size)`. A worsening attempt auto-reverts to the previous best. Termination is either the verified path or a `SAFE_STOP` artifact containing `{failure, evidence, attempted fixes, remaining`

`uncertainty, recommended human action}`. Every attempt is persisted and observable.

Implementation steps.

1. Loop controller with budgets and no-progress detection.
2. RCA generator (AI, schema, evidence-required) + deterministic heuristic fallback.
3. Hypothesis ranking.
4. Apply -> execute -> evaluate cycle reusing the editor and sandbox.
5. Candidate scoring + auto-revert.
6. Attempt ledger + state.
7. `SAFE_STOP` artifact.

Phase-wise testing.

- *Unit:* loop controller (max iterations, wall-clock, no-progress abort); candidate scorer / ordering; auto-revert trigger; `SAFE_STOP` artifact shape; RCA schema guard.
- *Integration:* a fixture with a deterministic one-line bug is repaired to green within budget and the ledger shows the attempts; an unfixable fixture ends in `SAFE_STOP` with evidence, no infinite loop, and the workspace restored to the best (or original) state.
- *Acceptance:* the acceptance repo with a seeded regression -> detect, investigate, repair, targeted + regression pass, verdict `VERIFIED`; the ledger reads like the Specification's example (Attempt 1 failed ... Attempt 3 passed).
- *Regression:* a property test that the loop never exceeds `max_iterations`; metric #5 and #11 on the seeded-fault set at or within target.

Quality gates. Repair attempts bounded; each iteration recorded; revert-on-worse works; safe stop returns evidence; no infinite loops (property-tested).

Metrics touched. #5 (primary), #11 (primary).

Risks & mitigations. Oscillation -> no-progress detector + best-candidate memory. AI thrash -> diff-size penalty + iteration cap. Long runtime -> wall-clock budget.

Effort. XL.

---

23. Phase 15 — Regression Intelligence

Goal. A regression-selection engine that classifies tests as `TARGETED`, `RELATED`, `REGRESSION`, or `FULL-SUITE` from changed files/symbols, the dependency graph, test-to-code links, previous failures, and affected modules. Supports both intelligent selection and full-suite execution. Correctness is never traded for speed: a full (or justified subset) run gates completion.

Depends on. Phases 5, 11, 12, 13.

Deliverables.

- `testing/regression.py`.
- Schemas: `RegressionPlan`, `RegressionResult`.
- API: `GET /tasks/{id}/regression`; state `REGRESSION_TESTING`.

Key design decisions. `TARGETED` = tests directly covering changed symbols; `RELATED` = tests within k hops / the same module / sharing fixtures; `REGRESSION` = historically-failed-in-area + high-centrality tests; `FULL` = everything. Policy: the repair loop uses `TARGETED + RELATED`; pre-verification requires `REGRESSION` then `FULL` (or a documented subset with a written justification and a risk note). A selection rationale is recorded per test. Previous-failure data comes from engineering memory (Phase 20) via a hook.

Implementation steps.

1. Coverage/graph-based test mapping.
2. Classifier + rationale.
3. Selection policies per stage.
4. Executor integration (batched to the sandbox) + result merge.
5. Full-suite runner + diff against baseline.
6. API + persistence.

Phase-wise testing.

- *Unit:* classifier on a fixture (changed symbol -> correct targeted set; k-hop related; centrality pick); per-stage policy selection; rationale attached.
- *Integration:* a fixture change -> the targeted set runs fast and correctly; the full suite runs and compares to baseline; a newly failing test is flagged as a regression.
- *Acceptance:* the acceptance task -> the regression suite executes with no unexpected failures and a selection rationale; forcing full-suite is also green.
- *Regression:* metric #6 on seeded regressions at or above target; selection determinism.

Quality gates. Tests classified with rationale; both smart and full modes work; regressions detected on the seeded set; the full-suite gate is enforced before verification.

Metrics touched. #6 (primary).

Risks & mitigations. No coverage data -> fall back to graph + naming with lower confidence. Flaky tests -> a quarantine list + a documented retry policy.

Effort. M–L.

---

24. Phase 16 — Code Review Engine

Goal. Automated review of every patch across ten categories — correctness, scope compliance, security, maintainability, architecture, performance, error handling, test quality, regression risk, dependency impact — producing `ReviewFinding` items with severity, category, file, line/range, description, evidence, recommendation, and confidence.

Depends on. Phases 10, 11.

Deliverables.

- `review/agent.py`, `review/static_checks.py` (ruff, bandit, radon, `mypy` on the diff), `review/rules.py` (custom AST rules), `review/aggregate.py`.
- Schemas: `ReviewFinding`, `ReviewReport`.
- Model: `ReviewFinding`.
- API: `GET /tasks/{id}/review`; state `REVIEWING`.

Key design decisions. Hybrid: deterministic static analyzers on the diff only (`bandit` for security, `ruff` / `mypy` for quality, `radon` for complexity, custom AST rules for test deletion, secret literals, `eval` / `exec`, `subprocess(shell=True)`, new dependencies) **plus** an AI reviewer (schema-constrained, evidence-required, must cite file + line). De-duplicate and normalize severity. Findings feed the PCS/CRS and the Scope Guard. Any AI finding without a concrete line is downgraded to `INFO`. Unresolved `CRITICAL` / `HIGH` gates the PR.

Implementation steps.

1. Integrate static analyzers scoped to the diff.
2. Custom AST safety rules.
3. AI review prompt + schema + evidence enforcement.
4. Aggregation / de-dup / severity model.
5. Report + API + state.
6. Wire to scoring and the scope guard.

Phase-wise testing.

- *Unit:* each static rule on positive/negative fixtures (secret literal, `eval`, bare `except`,

  test deletion, added dependency); severity normalizer; AI-finding evidence enforcement (no line -> `INFO`); de-duplication.
- *Integration:* a patched fixture containing an injected vulnerability (`subprocess(shell=True)`)

  -> a `HIGH` security finding; a clean patch -> no criticals.
- *Acceptance:* the acceptance patch is reviewed; findings are persisted with evidence and severity;

  no false `CRITICAL`; any real issue is cited with a line and a recommendation.
- *Regression:* metric #9 on the seeded-defect set at or above target; finding-schema snapshot.

Quality gates. Patch reviewed; findings persisted with evidence and severity; both static and AI paths run; criticals gate downstream.

Metrics touched. #9 (primary).

Risks & mitigations. Static-tool noise -> a curated, diff-scoped ruleset. Vague AI findings -> evidence gate + downgrade.

Effort. L.

---

25. Phase 17 — Risk & Confidence Engine

Goal. Implement the deterministic, documented, versioned Patch Confidence Score, Change Risk Score, Repository Health Profile, and Task-Specific Risk Profile from Section 4.10 — with explicit signals, normalization, weights, formula, thresholds, evidence, and `model_version`.

Depends on. Phases 8, 12, 14, 15, 16.

Deliverables.

- `scoring/confidence.py`, `scoring/risk.py`, `scoring/repo_health.py`,

  `scoring/model_registry.py` (versioned parameters).
- Schemas: `PatchConfidence`, `PatchRiskAssessment`, `RepositoryHealthProfile`.
- API: `GET /tasks/{id}/risk`, `GET /tasks/{id}/confidence`, `GET /repositories/{id}/health`.

Key design decisions. Formulas exactly as in Section 4.10. Parameters live in a versioned config (`scoring-model v1.0.0`). Each score returns `{value, classification, per_signal_contributions, evidence_refs, model_version}`. Hard gates (unresolved `CRITICAL`, scope violation, failing regression) override to `BLOCKED`. Calibration is deferred to Phase 25 with a dataset. No arbitrary numbers — every constant is documented, and a test asserts code == documentation.

Implementation steps.

1 Signal collectors (tests, review, scope, graph centrality, coverage, Git churn when present,

repair count).

1 Normalization functions with unit ranges.

2 Weighted aggregation + gates.

3 Classification thresholds.

4 Explanation payload (contribution breakdown that sums to the score).

5 Model registry + versioning.

6 API + persistence.

Phase-wise testing.

- *Unit:* each normalizer (bounds, monotonic); weighted-sum determinism; gate overrides;

  classification boundaries (85 / 70 / 50; 25 / 50 / 75); the explanation sums to the score; `model_version` stamped.
- *Integration:* an end-to-end task computes PCS / CRS from real signals; a failing regression

  forces `BLOCKED`; a large diff raises the CRS band.
- *Acceptance:* scores are reproducible across runs (same inputs -> same output); the breakdown

  explains the value; a test asserts the code constants equal the `docs/METRICS.md` table.
- *Regression:* golden score fixtures; a property test that adding a `CRITICAL` finding never

  raises the PCS.

Quality gates. Explicit, deterministic, explainable, testable, reproducible, versioned, and documented — each asserted by a test.

Metrics touched. Underpins #7, #10.

Risks & mitigations. Mis-calibration -> `v1.0.0` marked provisional, calibrated in Phase 25. Signal gaps -> documented defaults + lower confidence.

Effort. M–L.

---

26. Phase 18 — Verification

Goal. The Verification Agent decides whether the task is actually complete by checking the requested behaviour, the tests, the regression suite, patch consistency, scope, security, and plan alignment. A task is never complete merely because code was generated.

Depends on. Phases 9, 10, 11, 12, 14, 15, 16, 17.

Deliverables.

- `verification/agent.py` + per-criterion evaluators + trace generator.
- Schema: `VerificationResult` (per-criterion verdict + evidence + overall `{VERIFIED,

NOT_VERIFIED, PARTIAL}` + confidence).

- API: `GET /tasks/{id}/verification`; states `VERIFYING`, then `COMPLETED` / `AWAITING_APPROVAL` / `FAILED`.

Key design decisions. A deterministic checklist evaluator: (1) issue-specific acceptance checks derived from the plan's expected-behaviour section plus the generated issue-tests must pass; (2) targeted + regression + full-suite green (or justified); (3) the patch applies to a clean snapshot and is reversible; (4) the scope guard is clean or overridden-with-reason; (5) review has no unresolved `CRITICAL` / `HIGH`; (6) plan alignment — steps implemented, no unplanned files — from the Phase 10 traceability; (7) `PCS >= threshold` and `CRS <= threshold`, else `REVIEW_REQUIRED`. Overall `VERIFIED` only if every mandatory criterion passes. Produces the explainability trace (why file / why change / why test / why safe).

Implementation steps.

1. Criterion evaluators.
2. Plan-alignment checker (planned vs actual diff).
3. Aggregate verdict + confidence.
4. Explainability-trace generator.
5. API + state transitions.
6. Hook to the PR phase (only on `VERIFIED`).

Phase-wise testing.

- *Unit:* each criterion evaluator on pass/fail fixtures; alignment checker (unplanned file -> fail; missing step -> fail); aggregate logic (one mandatory failure -> `NOT_VERIFIED`); trace-generator shape.
- *Integration:* the full pipeline on the acceptance repo -> `VERIFIED` with all criteria and a trace; adding an unrelated file -> verification fails on scope and alignment; breaking a regression test -> `NOT_VERIFIED`.
- *Acceptance:* "code generated but tests failing" -> `NOT_VERIFIED`; the false-complete rate is 0 on the negative fixtures.
- *Regression:* metric #10 on the labeled set at or above target; trace snapshot.

Quality gates. Requested behaviour verified; regression evaluated; scope checked; plan alignment checked; no false-complete on the negative fixtures.

Metrics touched. #7, #10 (primary), #12.

Risks & mitigations. Weak acceptance-check derivation -> require an explicit expected-behaviour section in the plan and issue-tests. Over-strict verdicts -> a `PARTIAL` verdict + a human-review path.

Effort. L.

---

27. Phase 19 — Git / GitHub Integration

Goal. Deep Git intelligence (blame, history, churn, related commits, regression history) feeding planning, mapping, and review; a GitHub provider for repository / issue / PR / branch / commit retrieval and optional branch -> commit -> PR creation on `VERIFIED`. Tokens are never exposed; every failure mode is a structured state.

Depends on. Phases 3, 18; retroactively enriches 6, 7, 12, 16.

Deliverables.

- `git/history.py`, `git/blame.py`, `git/churn.py`.
- `github/client.py`, `github/provider.py`, `github/pr_builder.py`.
- Schemas: `GitContext`, `PullRequestDraft`.
- Models: `Commit`, `PullRequest`.
- API: `GET /repositories/{id}/git/*`, `POST /tasks/{id}/pr`, `GET /github/*`.

Key design decisions. GitPython for local history (no network). GitHub via REST with a token from the secret store, redacted in logs, optional. Write operations are gated by the HITL policy (protected branch / external PR -> `REVIEW_REQUIRED`). The PR body is built from the 18-section report structure (Specification Section 33): title, summary, issue reference, files changed, tests added, tests executed, results, review results, risk, confidence, known limitations. With no write credentials, only a local PR artifact is produced. A PR is claimed created only on an API 201 with a stored URL. URL validation plus rate-limit / network / permission / branch-conflict handling map to structured states.

Implementation steps.

1 Git history / blame / churn extractors + graph enrichment (`CHANGED_BY` / `FIXED_BY`).

2 History queries: "has this area changed?", "what happened after that change?", "was a similar

bug fixed before?", "which tests changed with similar modifications?".

1 GitHub client (optional auth, retries, rate-limit aware).

2 Issue import + repository fetch (ties to Phases 3 and 6).

3 Guarded branch / commit creation in the workspace + push.

4 PR builder + create + persist the URL.

5 Local PR artifact fallback.

6 Failure-mode mapping.

Phase-wise testing.

- *Unit:* churn / blame parsers on a crafted-history fixture; GitHub client against mocked HTTP

(201 -> PR stored; 403 -> permission state; 429 -> rate-limit state; network error; invalid URL); token redaction in logs.

- *Integration:* local history queries answer the four Specification questions on a fixture; the PR

builder produces an artifact with every required section; the no-credentials path yields only a local artifact and a state that reflects "not created".

- *Acceptance:* the acceptance task with credentials configured (mocked API) -> branch + commit + PR

"created" (mock 201) with the correct body; without credentials -> a local PR artifact; never a false "PR created".

- *Regression:* contract tests for the GitHub client; a secret-leak scan of logs; git-query golden

outputs.

Quality gates. Git history analysed; branch / commit creatable; a PR is created only after `VERIFIED` and only when credentials permit; tokens never exposed (asserted); failure modes structured.

Metrics touched. Enriches #1, #7; enables PR-related parts of #12.

Risks & mitigations. Live GitHub in CI -> fully mocked + one opt-in smoke test. Auth-scope errors -> clear messaging + `REVIEW_REQUIRED`.

Effort. L.

---

28. Phase 20 — Engineering Memory

Goal. Persist completed tasks (issue, repository, plan, implementation, tests, failures, root cause, repair attempts, final patch, review findings, verification) and retrieve relevant historical knowledge for future tasks — used as evidence, never as truth, never blindly copied.

Depends on. Phases 7, 9, 14, 16, 18.

Deliverables.

- `memory/store.py`, `memory/index.py` (FTS + optional embeddings), `memory/retrieve.py`.
- Schemas: `EngineeringMemory`, `MemoryHit` (similarity + provenance).
- Model: `EngineeringMemory`.
- API: `GET /memory`, `GET /tasks/{id}/memory`, `POST /memory/search`.

Key design decisions. A memory record is written at a terminal state (`VERIFIED` or `SAFE_STOP`). It is indexed by issue text, repository, touched symbols, failure signatures, and fix summary. Retrieval combines lexical + semantic + symbol overlap + a same-repository boost. Hits carry a similarity score and provenance. Consumers (Phase 7 mapping, Phase 9 planning, Phase 14 RCA, Phase 15 regression) receive memory as ranked evidence explicitly labelled "historical — verify". Old patches are never auto-applied. A repository-knowledge layer (structure, test/build commands, risky files, recurring failure patterns) is upserted each run.

Implementation steps.

1 Memory schema + a writer triggered on terminal states.

2 Repository-knowledge upserts.

3 Index build (lexical + optional embeddings).

4 Retrieval + ranking + provenance.

5 Integrate as optional, feature-flagged evidence into mapping / planning / RCA / regression, with

graceful behaviour when memory is empty.

1 API.

Phase-wise testing.

- *Unit:* writer serialization (all sections present); retrieval ranking (same-repo boost, symbol

overlap); empty-memory graceful path; the "historical — verify" labelling.

- *Integration:* run task A, then a similar task B -> B's mapping and planning receive A as an

evidence hit with similarity and provenance; A's patch is not auto-applied.

- *Acceptance:* the Specification's scenario ("Incorrect discount calculation" then "Discount limit

incorrectly applied") -> the prior task is retrieved as contextual evidence, clearly non-authoritative.

- *Regression:* retrieval determinism; disabling the memory flag reproduces pre-memory behaviour.

Quality gates. Task outcomes persisted with all sections; relevant retrieval works; memory used as labelled evidence, never blind-copied; graceful when empty.

Metrics touched. Improves #1, #5 over time.

Risks & mitigations. Stale or misleading memory -> provenance + recency weighting + a "verify" label + never authoritative.

Effort. M.

---

29. Phase 21 — FastAPI Completion + Job / Orchestration

Goal. Complete and harden every endpoint group and the job system, and wire the Orchestrator so the whole pipeline runs as one connected workflow driven by the state machine.

Depends on. All prior feature phases (this is the integration phase).

Deliverables.

- Full `api/*` routers for `/repositories`, `/tasks`, `/jobs`, `/analysis`, `/plans`, `/patches`, `/tests`, `/executions`, `/reviews`, `/verification`, `/github`, `/memory`, `/reports`.
- `orchestration/orchestrator.py`, `orchestration/job_queue.py`, `orchestration/state_machine.py`, `orchestration/worker.py`.
- Schemas: `Job`, `Timeline`. Complete OpenAPI. Minimal auth (API key / JWT).

Key design decisions. The Orchestrator drives every state from Section 4.3. A worker process (asyncio-based by default, abstraction allows RQ/Celery/arq) provides concurrency limits, idempotency keys, duplicate detection, retry with backoff, a cooperative cancellation flag, and crash recovery (resume from the last persisted step). Every transition is persisted and emitted to the timeline. Consistent problem-details error envelope. Cursor pagination and filtering on list endpoints. Auth guards on mutating and configuration endpoints.

Implementation steps.

1. Finalize the state-machine module with transition guards and persistence.
2. Job queue + worker (dedup, idempotency, retry, cancel, recover, concurrency cap).
3. Orchestrator wiring every agent in sequence, each consuming the prior stage's persisted output.
4. Complete routers + Pydantic models + errors + pagination/filtering.
5. Auth middleware.
6. OpenAPI polish with examples.
7. Timeline endpoint aggregating steps, jobs, and events.

Phase-wise testing.

- *Unit:* the transition table (legal transitions succeed, illegal ones are rejected); the job queue (dedup, idempotency replay, retry/backoff, cancel mid-flight, concurrency cap); pagination/filter validation; auth guards.
- *Integration:* submit a task -> the Orchestrator runs the full pipeline with the mock provider and the sandbox -> `COMPLETED`; cancel during `EXECUTING_TESTS` -> `CANCELLED` cleanly; kill the worker mid-run -> it resumes to completion; a test asserts each stage's input record equals the prior stage's output record (the connectedness invariant).
- *Acceptance:* OpenAPI validates; every Specification endpoint group is present and returns real data; the concurrency limit holds under a load test.
- *Regression:* full API contract snapshot; the end-to-end pipeline test becomes the new regression anchor.

Quality gates. One connected workflow (asserted by the stage-input provenance test); every stage consumes prior structured output; the job system is complete; states explicit; OpenAPI valid.

Metrics touched. #12; hosts the harness for all metrics.

Risks & mitigations. Worker infrastructure choice -> keep the abstraction, default to a simple asyncio worker. Partial-failure recovery complexity -> checkpoint after every phase.

Effort. XL.

---

30. Phase 22 — Frontend Dashboard

Goal. A professional React + TypeScript dashboard that exposes real backend data across 14 screens, with the main Task page showing the full stage pipeline and every stage inspectable, using polling or real-time status updates.

Depends on. Phase 21.

Deliverables.

- `frontend/src/{pages,features,components,services,hooks,types}` with a typed API client derived from OpenAPI.
- Screens: Repository Dashboard, Create Task, Task Execution, Planning View, Code Impact View, Implementation / Diff View, Test Execution View, Debugging Timeline, Review Findings, Verification Result, PR Summary, Engineering Memory, Repository Knowledge Graph, System Settings.
- A diff / patch viewer.

Key design decisions. Data-only (no mock data in the shipped app). TanStack Query with state-driven polling on active tasks; optional SSE/WebSocket for the timeline. The diff viewer (monaco or react-diff-view) shows files changed, additions/deletions, modified functions, risk, scope compliance, related tests, review findings, and the exact patch before approval. Graph visualization (Cytoscape / vis-network) from the subgraph API. HITL approve/block controls where policy is `REVIEW_REQUIRED`. Accessible, responsive, dark/light.

Implementation steps.

1. API client + types + auth.
2. App shell + routing + layout.
3. Repository Dashboard + Create Task.
4. The Task page pipeline component (Understanding -> ... -> PR) with a panel per stage.
5. Planning and Impact views.
6. Diff/Patch viewer + Test Execution + Debugging Timeline.
7. Review / Verification / PR Summary.
8. Engineering Memory + Knowledge Graph.
9. Settings (providers, policies, limits — no secret display).
10. Polling / real-time + error and loading states.

Phase-wise testing.

- *Unit (Vitest + RTL):* components render backend fixtures; the diff viewer parses a sample patch; the polling hook's interval logic; the graph adapter.
- *Integration (MSW-mocked API):* the task flow screens navigate and show data; a HITL approve triggers the correct call; the error envelope surfaces a friendly message.
- *E2E (Playwright, real backend with mock provider + sandbox):* create a task, watch it progress to

`COMPLETED`, inspect every stage, view the diff, view verification, view the PR summary; the knowledge graph renders.

- *Acceptance:* the dashboard shows the actual workflow for the acceptance task end to end; a test

  asserts every panel issues a backend call (no decorative data).

- *Regression:* the Playwright suite runs headless in CI; visual snapshots of key screens; axe

  accessibility checks.

Quality gates. Real backend data on every screen; the full pipeline is inspectable; the diff viewer shows the exact patch; E2E is green.

Metrics touched. None directly; makes all of them visible.

Risks & mitigations. E2E flakiness -> deterministic mock provider + a fixed sandbox fixture. Graph performance -> node cap + lazy expansion.

Effort. XL.

---

31. Phase 23 — Excel / Reporting

Goal. openpyxl import/export that shares the same domain services as the API and dashboard: bulk task/repository import, multi-sheet reports, and the structured 18-section task report.

Depends on. Phase 21 (plus data from Phases 12–18).

Deliverables.

- `reporting/excel_import.py`, `reporting/excel_export.py`, `reporting/report_builder.py`,

  `reporting/templates.py`.

- API: `POST /reports/import` (xlsx), `GET /reports/tasks/{id}.xlsx`, `GET /reports/tasks/{id}`

  (JSON), `GET /reports/metrics.xlsx`.

- Workbook sheets: Tasks, Execution Results, Tests, Failures, Repairs, Reviews, Risk, Verification,

  Engineering Metrics.

Key design decisions. Import parses a workbook and calls the same `TaskCreate` / `RepositoryRef` services with identical validation to the API; errors are reported per row (no silent skips). Export pulls from the domain repositories (no bespoke queries) into the nine sheets, with formula cells for metrics. The task-report builder emits the 18-section structure (Requirement ... Remaining limitations) as JSON and xlsx, reusing the verification, review, and scoring outputs. Large data is streamed (write-only mode).

Implementation steps.

1. Workbook schema (sheet/column contracts) + validation.
2. Importer -> domain services + a per-row error report.
3. Exporter for the nine sheets from the domain layer.
4. The 18-section task-report builder.
5. Metrics workbook (Section 40 metrics).
6. API endpoints.

Phase-wise testing.

- *Unit:* workbook parser (valid, missing column, bad type, extra sheet); each sheet exporter's

shape; report-builder section completeness; formula cells.

- *Integration:* import a fixture xlsx of five tasks -> five tasks created identically to the API

  path; run one, export xlsx -> sheets populated from real rows; a round trip (export then re-import a subset) is consistent.

- *Acceptance:* the acceptance task -> the task report has all 18 sections with real data; the

  metrics workbook reflects actually measured metrics (not invented).

- *Regression:* golden workbook snapshots (headers + sample rows); a test asserts the importer uses

  the same validation path as the API.

Quality gates. Excel uses real backend data and shared services; nine sheets plus the 18-section report exist; import validation parity with the API.

Metrics touched. Presents all 16.

Risks & mitigations. openpyxl performance on large sheets -> write-only mode + streaming. Schema drift -> contract tests.

Effort. M.

---

32. Phase 24 — End-to-End Controlled Repository

Goal. Build the acceptance test repository — multiple Python modules, dependency relationships, existing tests, real Git history, intentionally incomplete behaviour, at least one reproducible bug, at least one feature request, test gaps, a realistic architecture — and the full E2E acceptance test that drives AEGIS through all 18 workflow steps in Specification Section 39.

Depends on. Conceptually first (fixtures are used from Phase 3 on); the full E2E lands here once the pipeline is complete. A minimal version of this repo exists from Phase 3.

Deliverables.

- `test-repositories/aegis-acceptance/` — source + tests + a `.git` with crafted history.
- `docs/ACCEPTANCE_SCENARIOS.md`.
- `backend/tests/e2e/test_full_pipeline.py`.
- A seeded-fault set plus gold mappings and expected files.

Key design decisions. Domain: a simple billing / checkout app (`invoice.py` with `calculate_total`, discount logic, `checkout.py`, `order_service.py`, config) matching the Specification's worked examples. The Git history includes a prior related fix (to exercise Git intelligence and memory). Canonical tasks: (A) "Fix incorrect total when discount exceeds the configured maximum and add validation for the boundary condition" — a bug that must fully pass, including an introduced-then-repaired failure; (B) a feature request exercising file creation. Gold artifacts: expected files, expected symbols, expected tests, expected verification.

Implementation steps.

1. Author the modules with a realistic architecture and intentional gaps.
2. Write the existing tests, leaving a deliberate coverage gap.
3. Craft the Git history across several commits, including a related historical fix.
4. Define the seeded bug and feature scenarios with gold expectations.
5. Write the E2E test: ingest -> ... -> verify -> (mock) PR, asserting each of the 18 steps produced

   the expected structured output.

1. Write a negative E2E (an unfixable variant -> `SAFE_STOP`).

Phase-wise testing.

- *Unit:* a repo-fixture integrity check (imports resolve, baseline tests pass); gold-file schema validity.
- *Integration:* cross-stage data flow against this repo (each stage was already covered in its own phase).
- *Acceptance / E2E:* the full 18-step workflow succeeds for task A (including detect-failure -> investigate -> repair -> regression -> verify); task B creates new files and tests and verifies; the unfixable variant reaches `SAFE_STOP`.
- *Regression:* this E2E suite becomes the top-level regression gate for every later phase and runs in CI with the mock provider and the Docker sandbox.

Quality gates. The controlled repo has all nine required properties; the full workflow executes successfully; E2E is green in CI; the negative path is safe.

Metrics touched. #12 (primary); provides ground truth for #1, #2, #10.

Risks & mitigations. E2E runtime / flakiness -> deterministic mock provider, pinned sandbox image, infra-only retries. Overfitting AEGIS to this repo -> keep the Phase 25 benchmark set separate.

Effort. L.

---

33. Phase 25 — Benchmarking + Metric Calibration

Goal. A repeatable benchmark framework over a curated set of controlled tasks; record task, repository, expected files, expected behaviour, tests, outcome, time, iterations, patch quality, and failures; compute the 16 objective metrics (including cost per verified task, latency percentiles, the competitive resolution-rate delta against reference agents, and replay fidelity) with a documented definition / formula / inputs / interpretation / limitations / dataset for each; calibrate the scoring model.

Depends on. Phases 17, 18, 24.

Deliverables.

- `benchmarks/datasets/`, `benchmarks/runner.py`, `benchmarks/tasks.yaml`, `benchmarks/report.py`.
- Finalized `docs/METRICS.md` (with real numbers) and `docs/BENCHMARK_RESULTS.md`.
- A calibration script and, if adjusted, `scoring-model v1.1.0` parameters with a changelog.

Key design decisions. Benchmark tasks are curated mini-repos with seeded bugs/features and gold answers, disjoint from the acceptance repo. The runner executes AEGIS headless per task, captures raw signals, and writes JSON + xlsx. Metrics are computed by the explicit formulas in Section 6. Calibration fits normalization bounds and weights against labeled outcomes (patch accepted/rejected, verified/false-complete) using a held-out split. No invented numbers — measured values with confidence intervals and limitations. Regression thresholds for CI are set from the measured baselines.

Implementation steps.

1. Assemble the benchmark dataset with gold labels.
2. Headless runner + raw-signal capture.
3. The 12 metric calculators + their tests.
4. Report generator (JSON / xlsx / Markdown).

5 Calibration script with a train/test split, updated parameters, and a changelog.

6 A CI "benchmark smoke" subset; the full benchmark on demand / nightly.

Phase-wise testing.

- *Unit:* each metric calculator against hand-computed fixtures (mapping F1, alignment, validity,

pass rate, repair success, regression detection, acceptance rate, scope compliance, defect detection, verification accuracy including false-complete rate, mean iterations, completion rate); the report serializer.

- *Integration:* the runner over a three-task micro-benchmark produces metrics and a report,

deterministically with the mock provider.

- *Acceptance:* a full benchmark run produces `docs/BENCHMARK_RESULTS.md` with real measured

metrics and limitations; the calibrated scoring model is documented and version-bumped; results are reproducible.

- *Regression:* metric-calculator golden tests; a test that scans the published docs to ensure

numbers are generated, never hard-coded.

Quality gates. Benchmark repeatable; all 16 metrics defined and measured (not invented); the competitive delta (#15) is computed against at least two reference agents on the identical task set; scoring calibrated, versioned, documented; results reproducible.

Metrics touched. All 12 (defines and measures them).

Risks & mitigations. Small-dataset noise -> report confidence intervals, label provisional. Calibration overfit -> held-out split + simple models.

Effort. L.

---

34. Phase 26 — Security Hardening

Goal. Systematically close every threat in Specification Sections 18 and 34: command injection, path traversal, arbitrary host execution, malicious repositories and generated code, secret leakage, unsafe environment variables, insecure file handling, SSRF, unauthorized repository access, unsafe Git operations, and untrusted AI outputs. Validate all external inputs; never trust repository files, Git metadata, AI outputs, issue text, generated patches, or test code.

Depends on. All phases (a cross-cutting audit plus fixes).

Deliverables.

- Finalized `docs/SECURITY_MODEL.md`.
- `core/security/*` — input validators, a workspace path jail, a subprocess allowlist, a secret

scanner, an env allowlist.

- `backend/tests/security/*` — a threat-test suite.
- Dependency scanning (`pip-audit`) + an SBOM; `bandit` in CI as a gate.

Key design decisions. A central validation layer for every external input (repository URL, paths, task text, config, uploaded xlsx, GitHub payloads, AI JSON). All path operations go through a workspace-jail utility (resolve, then assert containment). No `shell=True` anywhere; a subprocess allowlist. A secret scanner runs on logs, artifacts, and prompt digests (build fails on a hit). An env allowlist into the sandbox. AI outputs are always schema-validated and treated as untrusted (no `eval` / `exec` of AI text; no path from AI without a jail check). An SSRF guard on all outbound calls (host allowlist, private-IP block). Git operations use safe flags (no hooks, protocol allowlist). Rate limiting and authorization on the API. An audit-log entry for every mutating or autonomous action.

Implementation steps.

1. Build a threat-to-control matrix and self-audit for gaps.
2. Implement / consolidate the validators, path jail, and subprocess allowlist.
3. Secret scanning in CI + a runtime log-filter test.
4. Consolidate the SSRF guard + tests.
5. Sandbox hardening review (seccomp, caps, no-new-privileges, digest-pinned image) + expanded escape tests.
1. Dependency audit + SBOM + pinning + a CI gate.
2. Authorization + rate limiting on the API.
3. Abuse ("pen-test style") tests.

Phase-wise testing.

- *Unit:* the path jail (`../`, symlink, absolute, UNC -> rejected); the command builder rejects injection; the URL guard (private IPs, redirects, DNS-rebind note); secret-scanner patterns; the env allowlist.
- *Integration:* a malicious fixture repository (install hook, network exfiltration in a test, fork bomb, writes to `/etc`, reads a seeded secret env var, a container-escape attempt) is fully contained with a structured failure and no secret exposure; a malicious AI output (path traversal in an edit operation, an `eval` payload) is rejected by the schema or the jail.
- *Acceptance:* the security test suite is green and gating; `bandit` / `pip-audit` are clean or triaged with a documented exception; no secret appears in any artifact or log (scanned).
- *Regression:* the security suite is a hard CI gate for all subsequent changes; every new abuse case becomes a permanent test.

Quality gates. Every Section 18/34 threat has a control and a test; inputs validated; no `shell=True`; secrets never exposed (scanned); sandbox escape and exhaustion tests pass; the dependency-audit gate is in place.

Metrics touched. #8 (scope compliance) hardened; security signal of PCS/CRS validated.

Risks & mitigations. Residual container-escape risk -> document the threat-model limits and name gVisor as a future option. Scanner false positives -> a documented allowlist with justification.

Effort. L–XL.

---

35. Phase 27 — Performance & Reliability

Goal. Meet the repository-size limits of Specification Section 37 gracefully; ensure long-running operations are jobs, not request-blocking calls; add timeouts, retries, backpressure, and concurrency caps; return `PARTIALLY_SUPPORTED` instead of crashing; run load and soak tests; guarantee resource cleanup and retention.

Depends on. Phases 21, 22.

Deliverables.

- `core/limits.py` consolidated (repo size, file count, file size, history depth, analysis duration, generated tests, AI context, patch candidates, sandbox runtime / memory / CPU).
- `backend/tests/perf/*`.
- Finalized `docs/EXECUTION_MODEL.md`.

- Cleanup / retention jobs.

Key design decisions. Every limit is configurable and enforced with a clear `PARTIALLY_SUPPORTED{reason}` (no crash, no silent truncation). Analysis and graph algorithms have time budgets with approximate fallbacks. The AI context builder truncates deterministically with provenance. Job concurrency plus queue backpressure. Circuit breakers around the AI and GitHub clients. Artifact retention and workspace garbage collection. Idempotent retries. A health / readiness endpoint plus basic metrics.

Implementation steps.

1. Consolidate limits and their enforcement points and the `PARTIALLY_SUPPORTED` plumbing.
2. Time budgets + approximate algorithms (centrality, mapping).
3. AI-context windowing + truncation policy.
4. Concurrency / backpressure + circuit breakers + retries.
5. Artifact / workspace GC + retention config.
6. Load + soak tests + profiling.
7. Metrics endpoint + dashboard notes.

Phase-wise testing.

- *Unit:* each limit check (under / over -> `SUPPORTED` / `PARTIALLY_SUPPORTED`); truncation

  determinism; circuit-breaker state transitions; the GC selects only eligible artifacts.
- *Integration:* an oversized repository fixture -> `PARTIALLY_SUPPORTED`, no crash, partial results

  returned; an AI timeout -> the job fails cleanly with a retry policy; concurrent tasks respect the cap; the workspace is cleaned after a task.
- *Acceptance / perf:* analysis of a mid-size real repository completes within the documented

  budget; a soak test (N tasks over M hours) shows no leak (containers, volumes, file descriptors, memory stable); throughput numbers are recorded.
- *Regression:* performance budgets as CI thresholds with a variance band; a leak check in the

  nightly soak.

Quality gates. All Section 37 limits enforced with `PARTIALLY_SUPPORTED` (no crash); long operations are async jobs; the soak test is leak-free; documented performance budgets met.

Metrics touched. None directly; keeps the metric harness runnable at scale.

Risks & mitigations. Performance variance in CI -> wide bands + nightly full runs. Approximate algorithms reduce accuracy -> label confidence + document.

Effort. M–L.

---

36. Phase 28 — Documentation & Release

Goal. Finalize every Phase 0 document to match the built system; write user, operator, and developer guides plus runbooks; produce the MVP `SUPPORTED` / `PARTIALLY_SUPPORTED` / `UNSUPPORTED` matrix; and sign off the 30-point final acceptance contract with linked evidence.

Depends on. All phases.

Deliverables.

- Finalized `docs/*` (Architecture, this plan's status, Tech Stack, Data Model, Security Model, MVP

Definition, AI Agent Design, Metrics, Execution Model, Repository Analysis).

- `README.md`, `docs/USER_GUIDE.md`, `docs/OPERATOR_GUIDE.md`, `docs/DEVELOPER_GUIDE.md`,

`docs/API_REFERENCE.md` (from OpenAPI), `docs/RUNBOOKS.md`, `docs/ACCEPTANCE_CONTRACT.md` (the 30-point checklist with evidence links), `CHANGELOG`, `LICENSE`.

Key design decisions. Documents are generated from or verified against the code where possible: the API reference from OpenAPI, the data model from the schema classes, the metric constants from a test-asserted sync with `docs/METRICS.md`. Every `SUPPORTED` claim links to a passing test or an acceptance artifact. No feature is claimed that is not implemented and verified. A quickstart (`docker compose up` -> run the acceptance task) is included and exercised in CI.

Implementation steps.

1. Reconcile each Phase 0 document with the final implementation.
2. Generate the API reference from OpenAPI.
3. Write the user / operator / developer guides and runbooks (incident, sandbox, provider outage,

GC).

1. Build the MVP support matrix with evidence links.
2. Fill `docs/ACCEPTANCE_CONTRACT.md`, mapping all 30 criteria to tests / artifacts.
3. Quickstart + demo script.
4. A final review pass against Specification Section 55 (Rules 1–20).

Phase-wise testing.

- *Unit / docs-CI:* a link checker; an OpenAPI-vs-implemented-routes diff (fails on drift); the

metric-constants sync test; a "no undefined feature" check (support matrix <-> tests).

- *Integration:* the quickstart script runs in CI (`docker compose up` -> health -> run the

acceptance task via the API -> `COMPLETED`) as a documentation-accuracy gate.

- *Acceptance:* all 30 final acceptance criteria demonstrably pass with linked evidence; the Rules

1–20 audit checklist is signed; the MVP matrix is accurate.

- *Regression:* the docs-CI gates (links, OpenAPI drift, metric sync, quickstart) become permanent.

Quality gates. Every Phase 0 document reflects reality; the API reference matches the routes; the MVP matrix is evidence-linked; the 30-point acceptance contract is satisfied; the quickstart works in CI.

Metrics touched. Publishes all 16 (from Phase 25).

Risks & mitigations. Documentation drift over time -> automated drift checks in CI. Overstated scope -> the support-matrix test.

Effort. M.

Appendix A — Traceability Matrix (Specification Section -> Phase)

Spec section	Topic	Owning phase(s)
0	Greenfield status	Phase 0
1	Core objective	Whole plan (Phases 6–20)
2	Core engineering loop	Phase 21 (wiring); each stage in its phase

Spec section	Topic	Owning phase(s)
3.1	Repository intelligence	Phase 4
3.2	Repository memory	Phase 20
3.3	Issue -> code intelligence	Phase 7
3.4	Change impact analysis	Phase 8
3.5	Engineering plan generation	Phase 9
3.6	Plan vs implementation traceability	Phases 10, 18
3.7	Autonomous implementation	Phase 10
3.8	AI provider abstraction	Phase 0 (design), Phase 9 (delivery)
4	Multi-agent architecture	Phase 0 (design), Phase 21 (orchestrator)
5	Agent responsibilities	Phases 4, 9, 10, 11, 14, 16, 18
6	Test generation engine	Phase 11
7	Autonomous debugging loop	Phase 14
8	Characterization / baseline testing	Phases 15, 18
9	Regression intelligence	Phase 15
10	Code review engine	Phase 16
11	Scope guard	Phases 10 (tracker), 16, 18
12	Patch risk + confidence engine	Phases 0 (formula), 17
13	Engineering risk score	Phases 0 (formula), 17
14	Repository health profile	Phase 17
15	Git intelligence	Phase 19
16	GitHub integration	Phase 19
17	Pull request generation	Phase 19
18	Secure code execution	Phases 0 (threat model), 12, 26
19	Job orchestration	Phase 21
20	AI output schemas	Phase 0 (definition), each consuming phase
21	AI hallucination control	Cross-cutting; enforced Phases 7, 9, 13, 14, 16
22	Knowledge graph	Phase 5
23	Engineering memory	Phase 20
24	Explainability	Phase 18 (trace); recorded throughout
25	Observability	Phases 2 (logging), 21 (timeline)
26	FastAPI backend	Phases 2 (base), 21 (completion)
27	Frontend dashboard	Phase 22
28	Diff / patch viewer	Phase 22
29	Human-in-the-loop safety	Phase 0 (policy), Phases 9, 19, 21
30	Rollback	Phases 3, 10, 14
31	IDE-like task experience	Phase 22
32	Excel integration	Phase 23
33	Reporting	Phase 23
34	Security	Phase 26 (cross-cutting)

Spec section	Topic	Owning phase(s)
35	Database	Phases 2 (base), then per-phase migrations
36	Artifact storage	Phases 0 (design), 2, 12
37	Repository size limits	Phases 3, 27
38	MVP definition	Phase 0, revisited Phase 28
39	End-to-end acceptance repository	Phase 24
40	Objective metrics	Phases 0 (definitions), 25 (measurement)
41	Benchmarking	Phase 25
42	Failure modes	Every phase's failure-state handling; audited Phase 26/26
43	No fake functionality	Cross-cutting rule; enforced by tests everywhere
44	Development stack	Phase 0, Phase 2
45	Project structure	Phase 0, Phase 2
46	Phase 0 planning docs	Phase 0
47	Claude execution protocol	Every phase (Definition of Done)
48	Objective acceptance contract	Every phase's testing block
49	Quality gates	Every phase's quality-gate block
50	Phase-wise implementation	This whole document
51	Early architectural decisions	Phase 0 (ADRs)
52	Design principle (pipeline not chatbot)	Phase 21 connectedness invariant
53	Final end-to-end system	Phases 21, 24
54	Final acceptance criteria	Appendix C
55	Absolute rules	Appendix D
56	Ultimate objective	The whole plan

Beyond the Specification. This plan adds material the Specification does not mandate but the project needs to be competitive: strategic positioning and the wedge (Section 2.1–2.4, Phase 0 `POSITIONING.md`); the cost and latency budget with model routing (Section 4.13, `COST_MODEL.md`, Appendix F); trust, governance, deterministic replay, RBAC, and the Trust Report (Section 4.14, `GOVERNANCE.md`); the capability spike and walking-skeleton delivery strategy (Sections 7.1–7.2, Phases 0 and 1); the kill / decision gates (Section 7.5, Appendix E); and metrics #13–#16 with the competitive baseline (Section 6, Phase 25).

Appendix B — Milestones, Sequencing, and Critical Path

Milestone	Phases	Entry condition	Exit condition
M0 Capability spike & walking skeleton	0, 1	plan approved	capability floors met (or documented re-scope); one real task runs REQUIREMENT -> VERIFIED DIFF headless; connectedness asserted
M1 Repository understanding	2–5	M0 gates pass	a real Python repo is ingested, analyzed, and graphed; golden snapshots stable
M2 Planning	6–9	M1 gates pass	task -> mapping (evidence) -> impact -> validated plan; validator blocks bad plans

Milestone	Phases	Entry condition	Exit condition
M3 Implement + execute	10–12	M2 gates pass	real patch + generated tests + secure sandbox execution; escape tests pass
M4 Debug + regression	13–15	M3 gates pass	bounded repair to green on a seeded bug; regression selection with rationale
M5 Review + score + verify	16–18	M4 gates pass	review findings, deterministic PCS/CRS, verification verdict with trace; no false-complete
M6 Git + memory	19–20	M5 gates pass	Git intelligence answers the four questions; PR only after VERIFIED; memory retrieval works
M7 Product surfaces	21–23	M6 gates pass	connected pipeline via the API; dashboard shows real data end to end; Excel uses shared services
M8 Prove it	24–25	M7 gates pass	full 18-step E2E green; 16 metrics measured; competitive delta computed; scoring calibrated
M9 Harden	26–28	M8 gates pass	security suite gating; soak leak-free; 30-point acceptance contract signed

Critical path. Phase 0 (incl. capability spike) -> 1 (walking skeleton) -> 2 -> 3 -> 4 -> 7 -> 8 -> 9 -> 10 -> 11 -> 12 -> 13 -> 14 -> 15 -> 16 -> 17 -> 18 -> 21 -> 24 -> 25 -> 26 -> 28. Phases 5, 6, 19, 20, 22, 23, 27 have some slack; the task-pipeline view of 22 gates M7 completion, the full 14-screen dashboard does not.

Highest risk / front-load. (1) The **capability spike** in Phase 0 — if issue->code localization and end-to-end repair cannot clear the floors, nothing downstream matters. (2) **Phase 12** (Docker sandbox resource controls + escape prevention) — run a throwaway spike inside Phase 2 on the real CI runner. Next: Phase 14 (loop termination and auto-revert correctness), Phase 21 (crash-recovery and the connectedness invariant), Phase 22 (E2E stability).

Appendix C — Final Acceptance Contract (Specification Section 54)

Each criterion is satisfied by the listed phase and demonstrated by that phase's acceptance/E2E test, all re-run in the Phase 24 end-to-end suite.

#	Criterion	Satisfied in
1	A real repository can be ingested	Phase 3
2	Repository structure can be analyzed	Phase 4
3	Symbols and dependencies can be identified	Phases 4, 5
4	Git history can be analyzed	Phase 19
5	A real issue can be submitted	Phase 6
6	Relevant code can be identified	Phase 7
7	Evidence-backed impact analysis can be generated	Phase 8
8	A structured engineering plan can be created	Phase 9
9	The plan can be validated	Phase 9

#	Criterion	Satisfied in
10	Real code can be modified	Phase 10
11	Tests can be generated	Phase 11
12	Tests can execute in a sandbox	Phase 12
13	Failures can be detected	Phases 12, 13
14	Root causes can be investigated	Phases 13, 14
15	Repair attempts can be performed	Phase 14
16	Regression tests can execute	Phase 15
17	Generated changes can be reviewed	Phase 16
18	Scope violations can be detected	Phases 10, 16, 18
19	Risk can be calculated	Phase 17
20	Confidence can be calculated	Phase 17
21	Final verification can be performed	Phase 18
22	A real patch / diff can be produced	Phase 10
23	Git branch / commit can be created	Phase 19
24	GitHub PR can be created when credentials permit	Phase 19
25	Engineering memory can persist the task	Phase 20
26	Dashboard can show the actual workflow	Phase 22
27	FastAPI exposes the workflow	Phases 2, 21
28	Excel reports use real backend data	Phase 23
29	Failures are handled safely	Phases 14, 26, 27 (audited)
30	No fake functionality is used	Cross-cutting; enforced by tests in every phase

---

Appendix D — Absolute Rules Audit (Specification Section 55)

Rule	Where enforced
1 No disconnected modules	Phase 21 connectedness-invariant test
2 Don't optimize for agent count	Seven agents fixed in Phase 0; ADR
3 AI code is not automatically correct	Phases 12, 16, 18 (execute + review + verify)
4 Every implementation executed and tested	Phases 11, 12, 15; Definition of Done
5 Every AI conclusion has evidence	Schema `evidence` field; Phases 7, 9, 13, 14, 16 tests
6 Every score has an explicit algorithm	Section 4.10; Phase 17 code-vs-doc test
7 Never claim a test passed unless it executed and passed	Phase 12 status tracking; `INVALID`/`Executed` fields
8 Never claim a PR exists unless created	Phase 19 (only on API 201 + stored URL)
9 Never execute untrusted code on the host	Phase 12 sandbox; host subprocess spy tests
10 Never expose credentials	Phase 2 redaction; Phase 26 secret scanner (CI gate)
11 Every repair loop bounded	Phase 14 iteration + wall-clock + no-progress; property test
12 Every patch reversible	Phases 3, 10, 14 (baseline commit + rollback)
13 Scope expansion detected	Phase 10 scope tracker; Phases 16, 18
14 Execution results outrank AI	Section 4.7; Phases 12–14 precedence
15 Don't hide uncertainty	`UNKNOWN` / `LOW CONFIDENCE`; confidence fields everywhere

Rule	Where enforced
16 Don't fabricate metrics	Phase 25 (generated, not hard-coded); docs-scan test
17 No critical features as TODOs	Definition of Done; CI check for stub markers
18 Don't stop at architecture / scaffolding	This plan; every phase ends in verified functionality
19 Not a presentation prototype	Phase 22 "every panel calls the backend" test
20 Optimize for working integrated functionality	Milestone exit gates; Phase 24 E2E

Appendix E — Kill Criteria and Decision Gates

Autonomous software engineering is a capability bet before it is an engineering-scale bet. These gates make the bet falsifiable: at each one, a metric is compared to a threshold and a decision is forced. They exist so the project does not spend months building product surfaces on top of a core that cannot localize issues or repair code.

Gate	When	Metric(s)	Threshold	Rationale	Action if missed
G0	End of Phase 0 capability spike (Stage A)	issue->code localization recall@10; end-to-end verified-fix rate on the ~30-task subset	`>= 0.75`; `>= 0.30`	If AEGIS cannot find the right files and cannot fix a meaningful fraction of easy tasks with naive wiring, better wiring will not save it.	Up to two bounded retrieval/planning redesign rounds. Still below -> re-scope: assisted (human-in-loop suggestions) instead of autonomous, or a narrower task class (e.g. dependency bumps, lint fixes) where the floor is reachable.
G1	M2 exit	localization recall@10 on the **held-out** "wild" set; plan-validation reject rate on deliberately bad plans	`>= 0.70`; `>= 0.95`	Guards against overfitting to the curated set and against a validator that rubber-stamps.	Pause all breadth work; invest exclusively in mapping + planning + validation until recovered.
G2	M5 exit	false-complete rate on the labeled verification set	`<= 2%`	A system that says "done" when it is not is worse than no system — it destroys the trust that is the entire wedge.	Do not proceed to product surfaces. Tighten verification criteria (more mandatory checks, stricter plan-alignment, require issue-tests).
G3	M8 exit	competitive resolution-rate delta (#15); cost per verified task (#13); replay fidelity (#16)	quality delta `>= 0`; cost within `docs/COST_MODEL.md` envelope; fidelity `>= 0.9`	Release readiness: at least matches open reference agents on verified-change quality, is economically sane, and is auditable.	Hold release; fix the failing dimension (quality: retrieval/planning/review; cost: model routing + caching + early-exit; fidelity: seed capture + reduce nondeterministic prompts).

Gate decisions and their evidence are recorded in `docs/DECISIONS/` as dated ADRs.

Appendix F — Cost Model Worksheet

Illustrative per-task envelope for the MVP pipeline on a mid-size Python repo. Call counts are expected values; token estimates assume the deterministic context-windowing policy of Section 4.13. Prices are placeholders from the frozen priced-model table (`docs/COST_MODEL.md`, versioned) — recompute when prices change; the CI cost-regression gate (Section 5.7) uses the same table.

Stage	Tier	Calls (exp.)	~Input tok/call	~Output tok/call	Notes
Task normalization	cheap	1	1k	0.3k	classify + structure the issue
Issue->code mapping (rerank)	cheap	2–4	3k	0.5k	rerank lexical/graph candidates; embeddings are a one-off index cost per snapshot
Impact summary	cheap	1	3k	0.6k	
Engineering plan	frontier	1 (+1 repair)	8k	2k	schema-constrained; one repair round budgeted
Implementation edit-ops	frontier	1–2	10k	3k	anchored edit operations
Test generation	frontier	1–2	6k	3k	
Root-cause analysis (per repair iter)	frontier	0–2	8k	1.5k	bounded by Phase 14 iteration cap
Repair edit-ops (per repair iter)	frontier	0–2	9k	2k	
Code review (AI pass)	frontier	1	9k	2k	static checks are free
Verification reasoning	cheap	1	4k	0.8k	criteria are mostly deterministic

Envelope. Low (no repair needed): a handful of cheap calls + ~4 frontier calls. Expected (1–2 repair iterations): ~6–8 frontier calls + ~6 cheap calls. High (full repair budget): capped by the per-stage AI-call budgets and the repair iteration + wall-clock bounds — the Orchestrator refuses to exceed them and parks the task instead. The concrete USD figures live in `docs/COST_MODEL.md` so this plan does not carry numbers that drift.

Levers if cost/latency regress: tighten model routing (push more stages to `cheap`), raise the context-windowing aggressiveness, lower the repair iteration cap, enable/extend prompt caching for the repository-context prefix, and turn on the marginal-improvement early-exit.

Appendix G — Regenerating this Document

This Markdown file is the editable source. To rebuild the PDF:

```
python scripts/build_plan_pdf.py
```

The script reads `docs/AEGIS_IMPLEMENTATION_PLAN.md` and writes `docs/AEGIS_IMPLEMENTATION_PLAN.pdf` using reportlab. Edit the Markdown, then re-run the script.

End of plan.